20 September 2024

Constraint Random Verification

Harry Duque
harry.duque@axis.com

Malin Heyden
malin.heyden@axis.com

Agenda

- About me
- Background
- Constraint Random Verification
- Case study
- Final remarks



About me

- Bachelors in EE from Universidad Simon Bolivar, Venezuela, 2003.
- Master in System-On-Chip from LTH, Sweden, 2005.
- Over 18 years of experience in ASIC industry.
  - Frontend
  - Power Management
  - Functional Verification



About me – Malin Heyden

- E16
- IC-master plus some software
- Master Thesis 2021 High Level Synthesis
- At Axis since 2021 – Asic Verification Engineer
- Mostly module level verification, mostly simulation, some formal



Axis Communications

> Leader in network video surveillance

> Swedish company

> Part of Canon group

> About 4000 employees worldwide







Axis ASIC Development

- Long history of chip development
  - TGA/CGA – Printer connectivity
  - ETRAX – SoC for network connected devices
  - ARTPEC – Network video
- Why in house chip development?
  - Differentiation
  - Optimization for application
  - Access to technology
  - Unit cost



Background

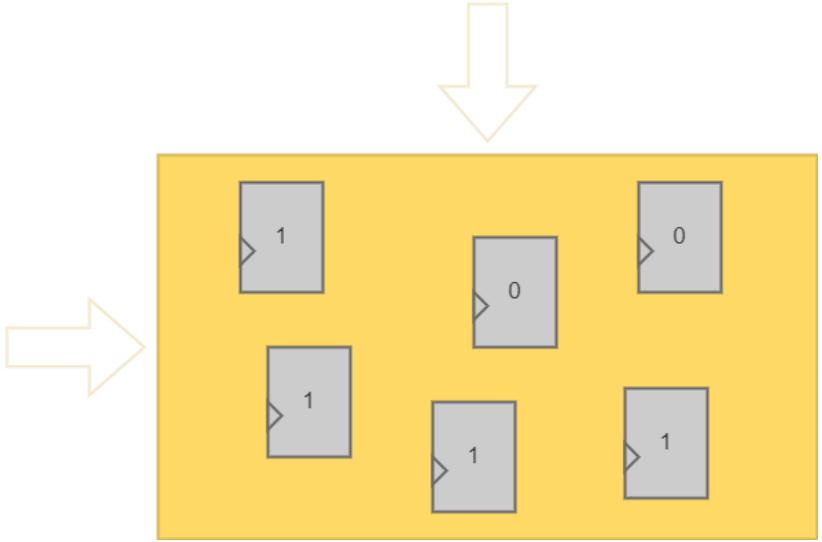
Scope

- Functional Verification
  - Dynamic
    - Simulation
      - Directed testing
      - **Constraint Random Verification (a.k.a. Coverage-Driven, Constraint Random)**
      - …
    - FPGA prototyping
    - …
  - Static
    - Lint
    - Formal
    - …



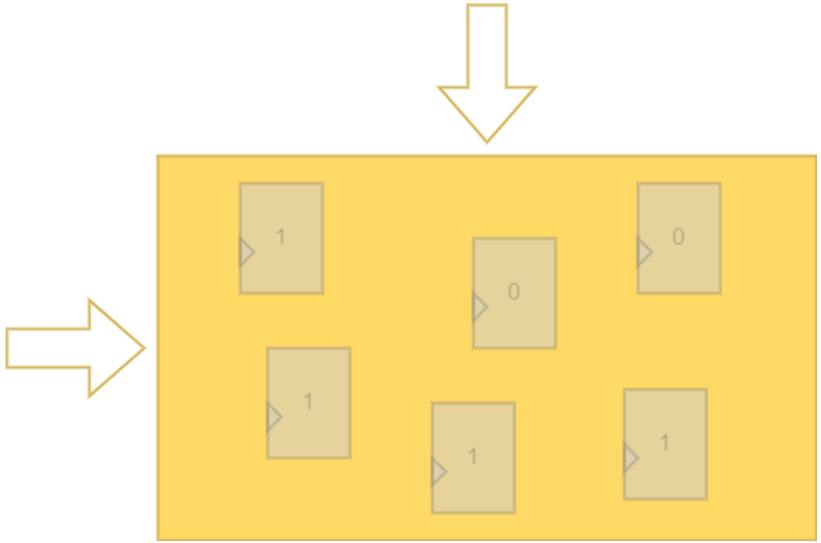
State Space

- Internal state space
  - Also known as design state space.
  - Defined by the states of all sequential elements in the design, i.e. memories and flip-flops.
  - With **n** flip-flops the internal space size is **2**n**



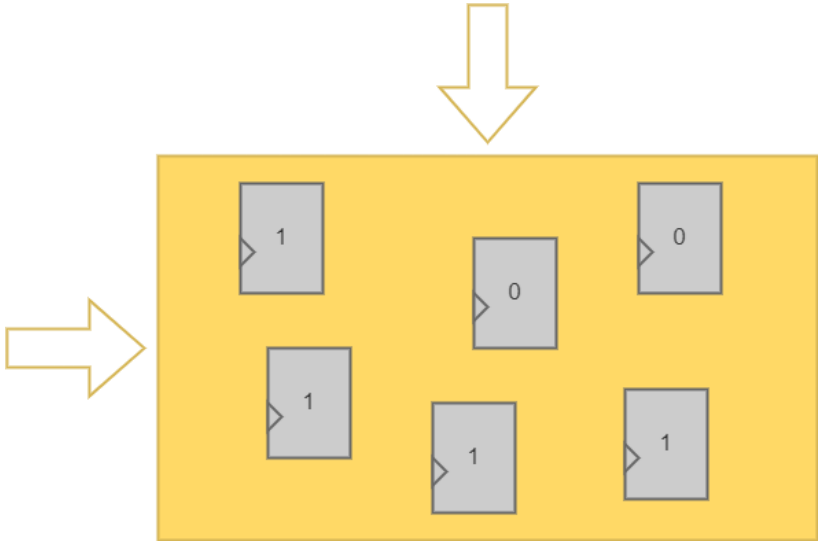
State Space

- External state space
  - Also known as stimulus space.
  - Defined by all the possible combinations of inputs to the design.
  - With **m** binary inputs the external space is **2**m**



State Space

- Total state space
  - The overall state space is then the product of the external and the internal state space.
  - **space = 2**n x 2**m**
  - Not all space is reachable.
- To exhaustively verify such a design, all reachable state space need to be tested.



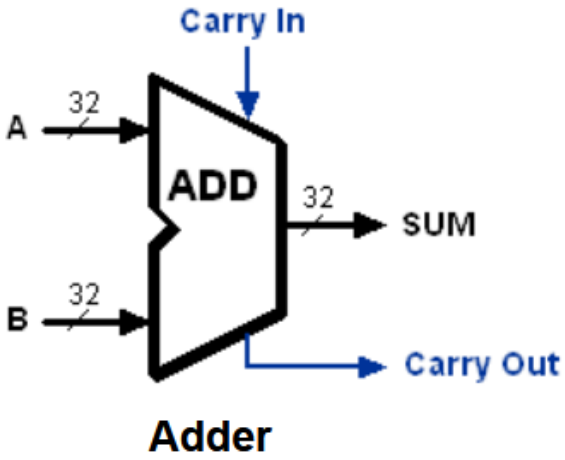
A text-book example

> Design: Two-input 32-bit adder.

> Purely combinatorial design.

> State space consists of 2**65 different input combinations

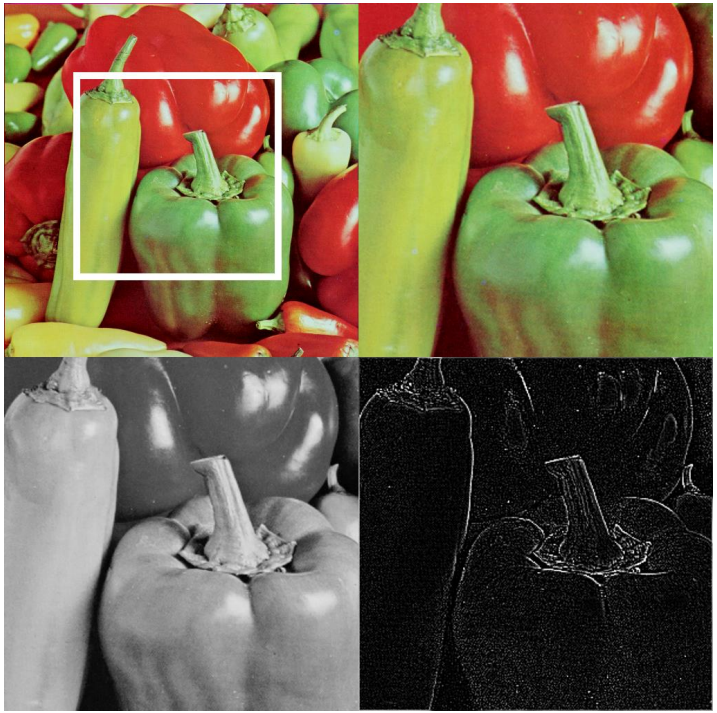
> At 2**20 cycles per second (fast!) it will take **2**45** seconds.



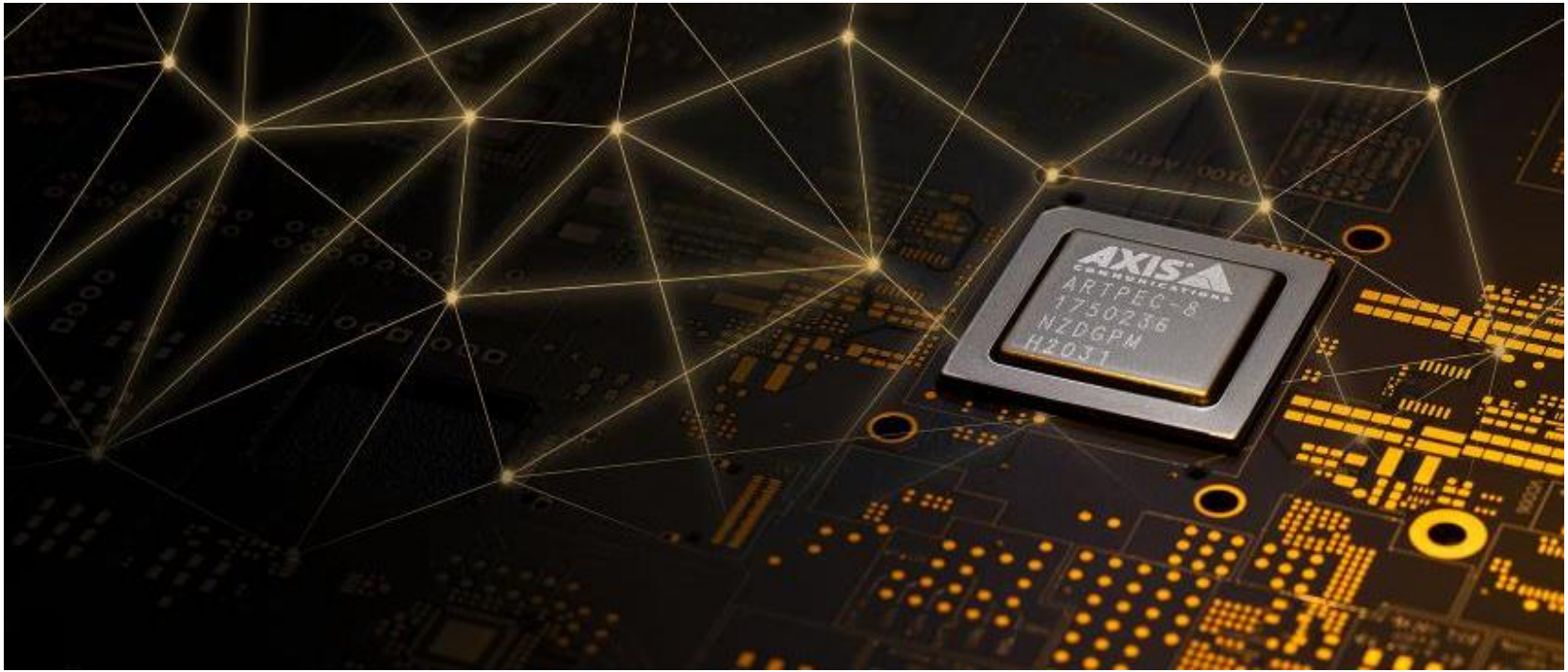
Adder

A more complex example

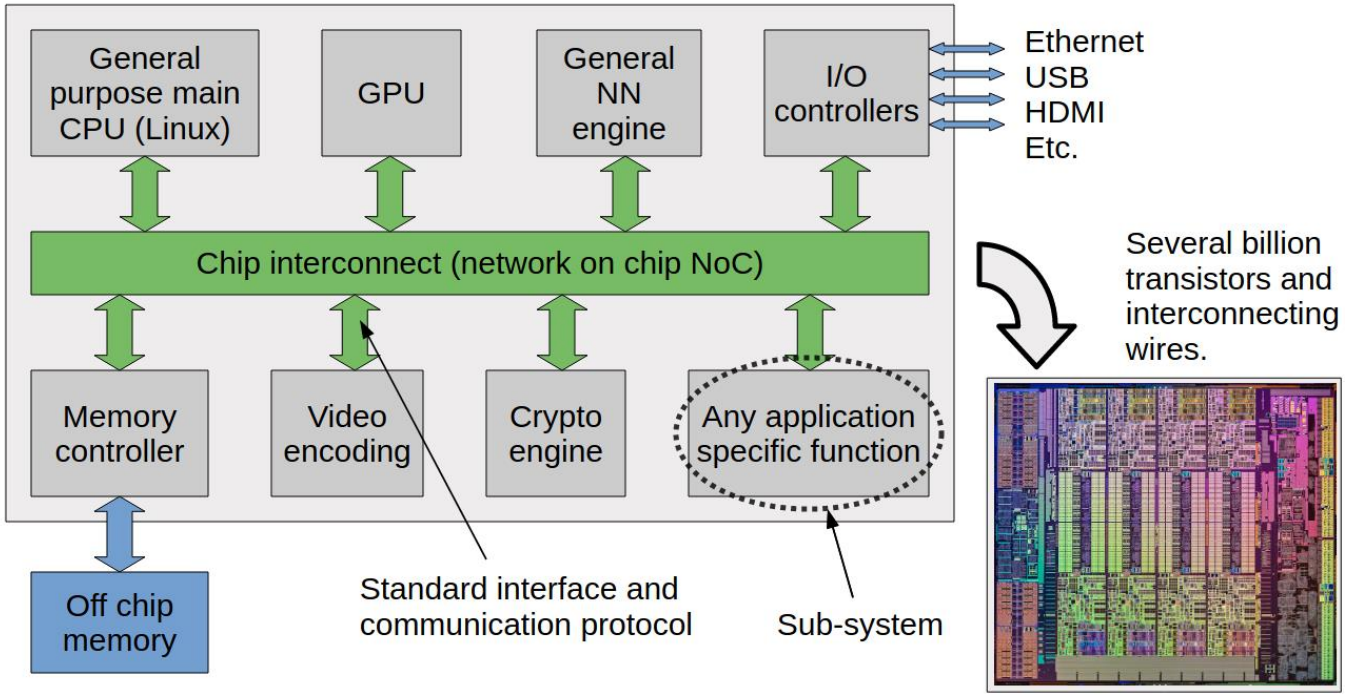
- Design: Edge detection in image pipeline
- Design state space:
  - Flops: 685
- Stimulus state space
  - 34-bits of configuration
  - 24-bit RGB data.
- Total: 2**685 x 2**58 = 2**743



How about a SoC?

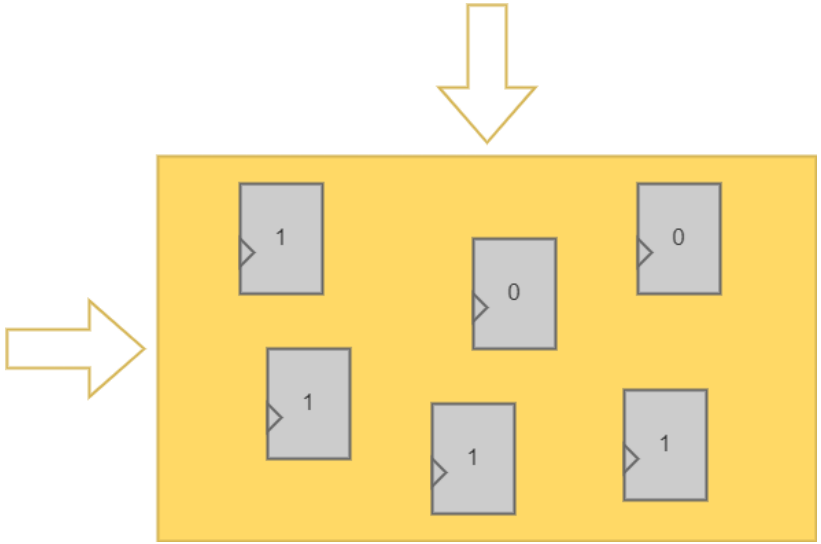


ARTPEC – Axis Real Time Picture Encoding Chip



Goals of ASIC verification

> Make sure that the design meets the system requirements and specifications.
>
> – **Correctness**
>
> – **Completeness**



How do we go about it?

- Divide and Conquer.
- Do not treat design as a black-box, leverage on:
  - Knowledge of design internal structure.
  - Corner cases.
  - Orthogonality.
- Also consider,
  - Risk.
  - Previous experience.

One way. Directed testing

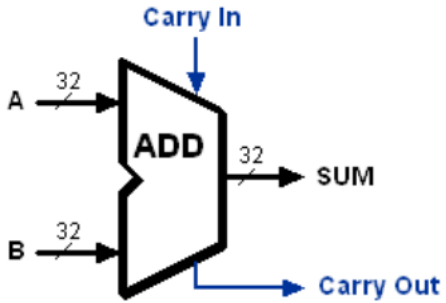
> Verification goal is to define a set of tests, that gives an acceptable confidence level in the verification.

> Each test:
  - Is itself the definition of an individual, relevant scenario to observe (**metrics**).
  - Contains the respective test input vector (**stimulus**).
  - Contains manually pre-computed *expected* output.
  - Is simulated, and the DUTs *actual* output is gathered.
  - Compare *expected* vs *actual* (**checking**).



Back to our text-book example

- Homogenous structure.
  - Chain of full adders.
- Example of verification plan with directed testing.
  - Walking pattern of 1’s
  - All zeros
  - All ones
  - Overflow
- A few dozen tests.
- Relatively easy to convince ourselves that the design is likely bug-free.



Adder

How about our more complex example?

Directed testing

> Consider …
  - Re-use
  - Separation of concerns
  - Verification confidence
> Problems ...
  - Effort
  - Error prone.
  - High maintenance
  - What about any unthought-of-cases?
  - **Scalability**



Effort vs risk.

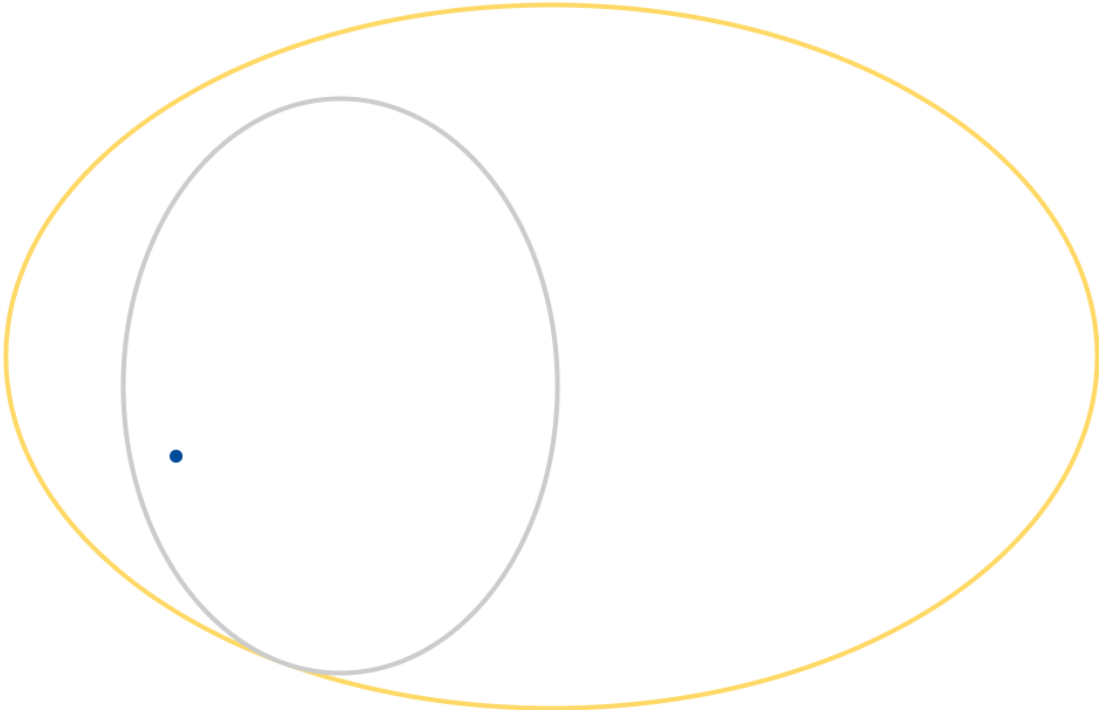




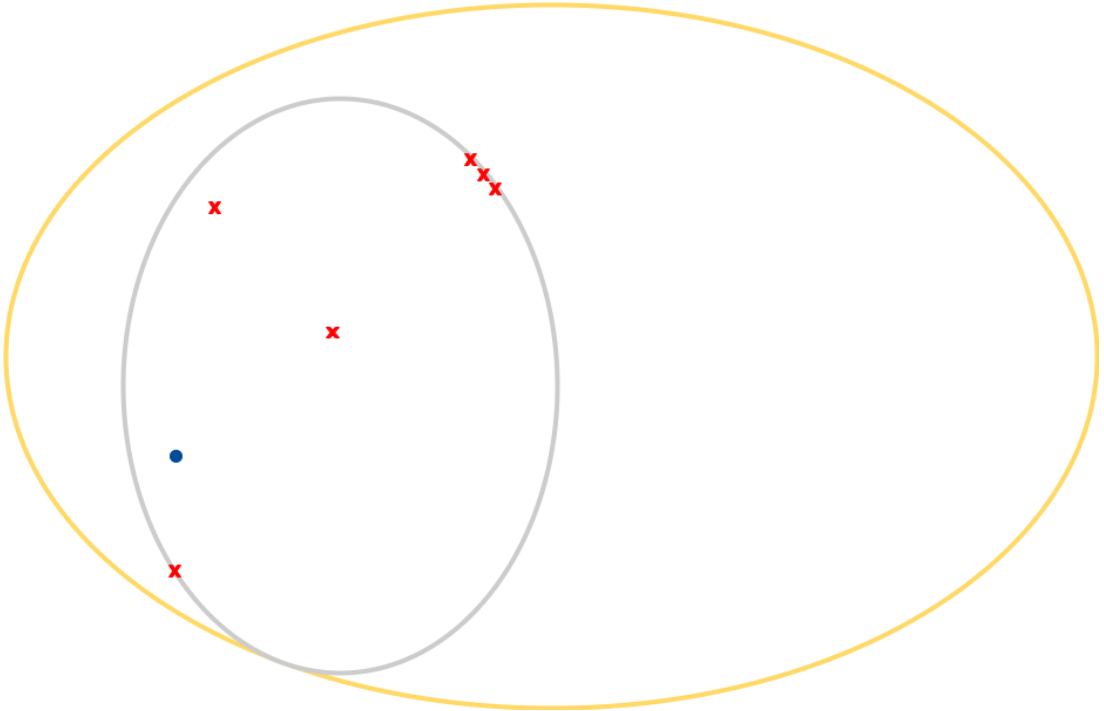


Constrained Random Verification (CRV)

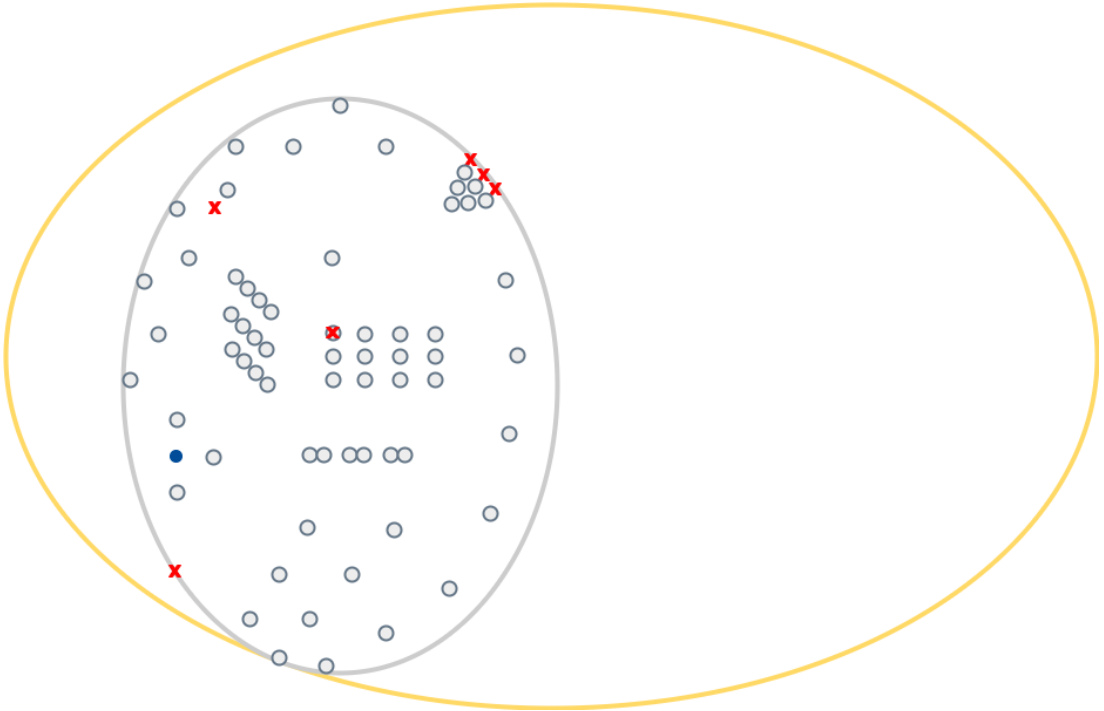
The basics



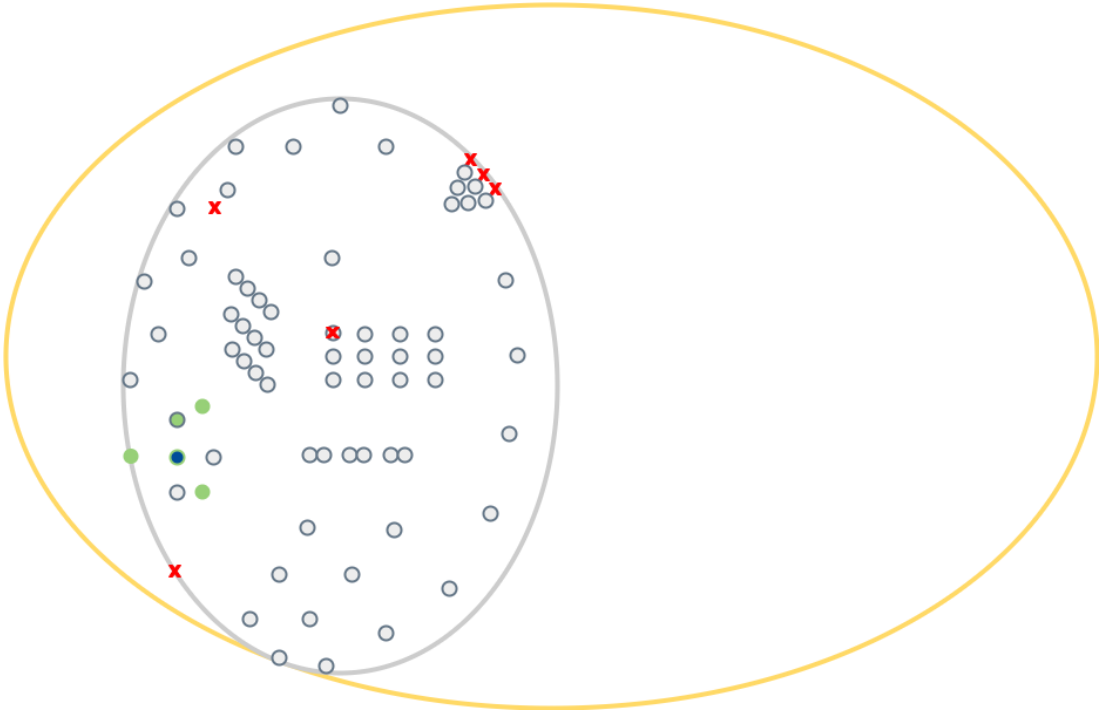
The basics



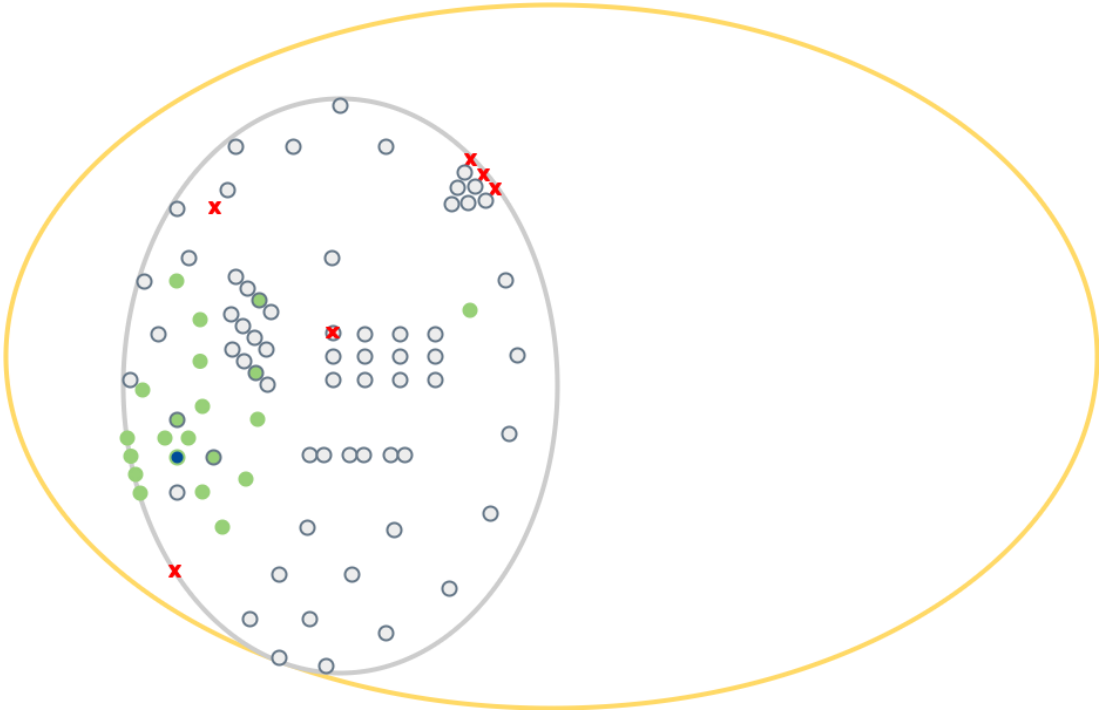
The basics



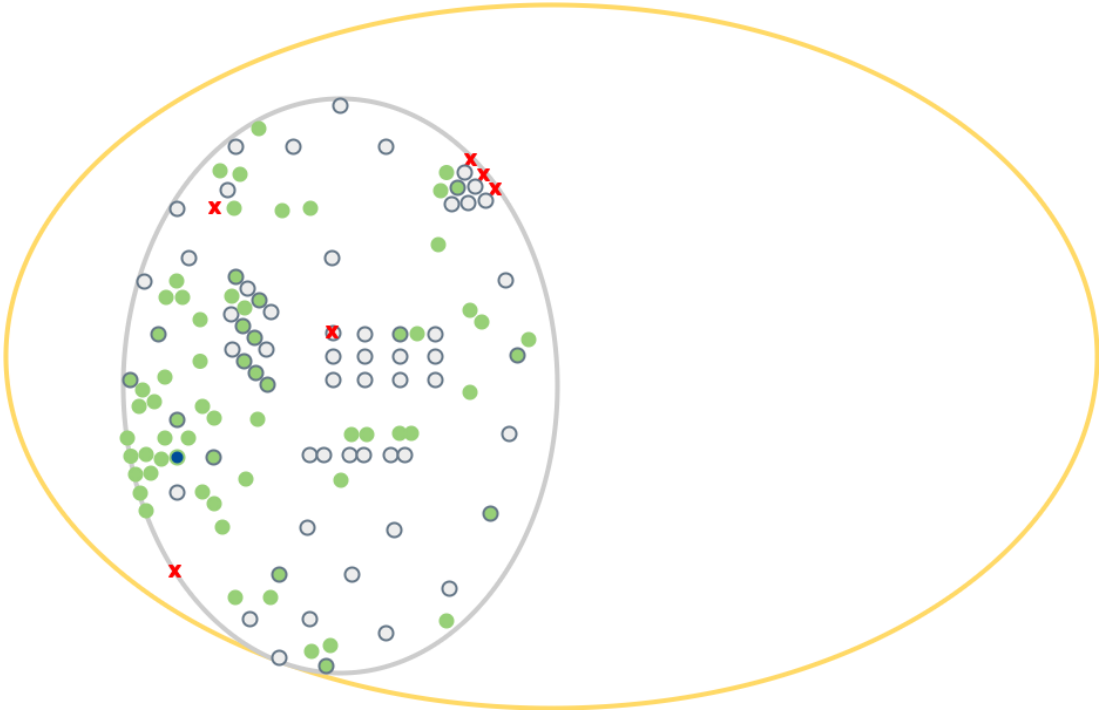
The basics



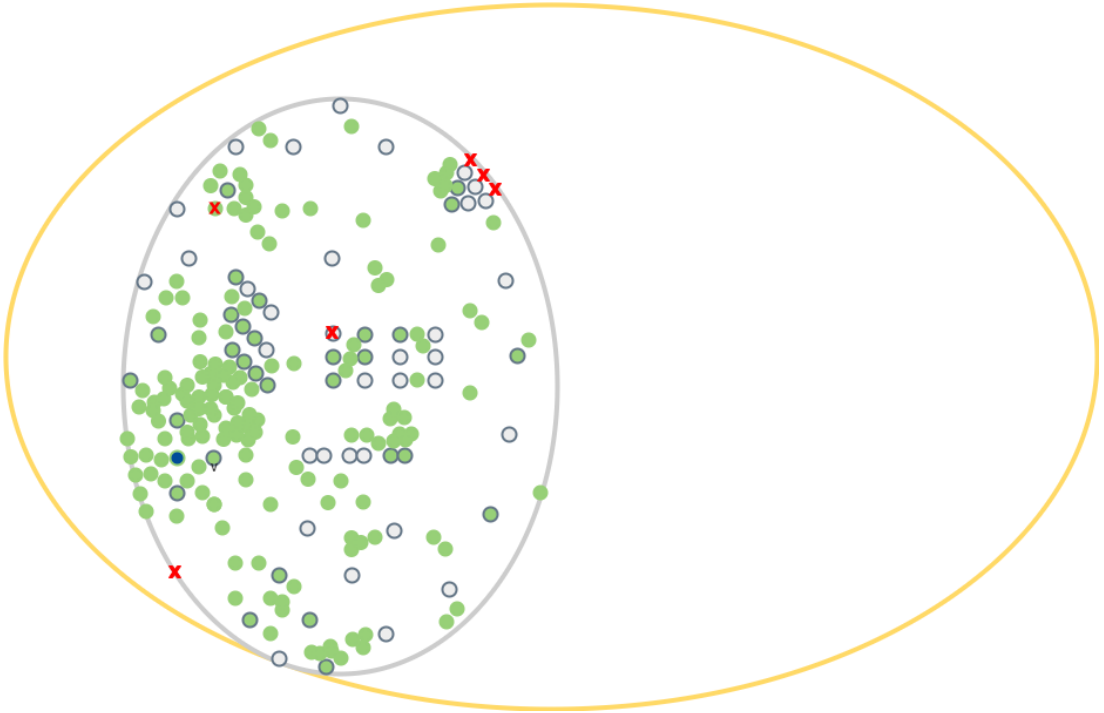
The basics



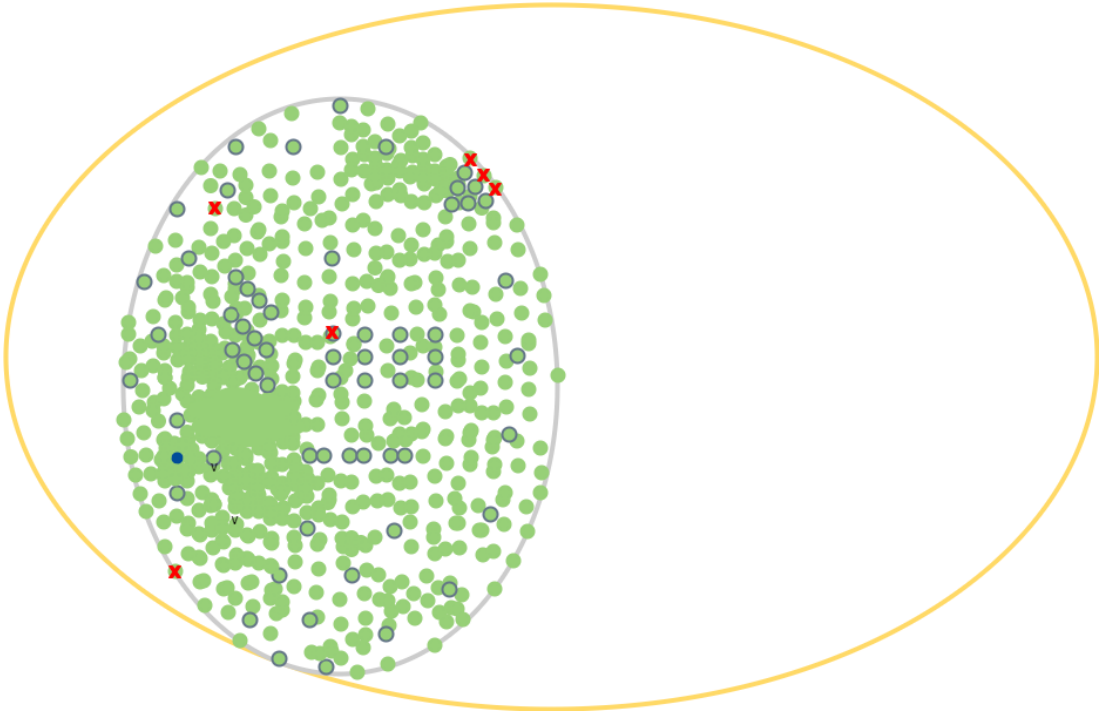
The basics



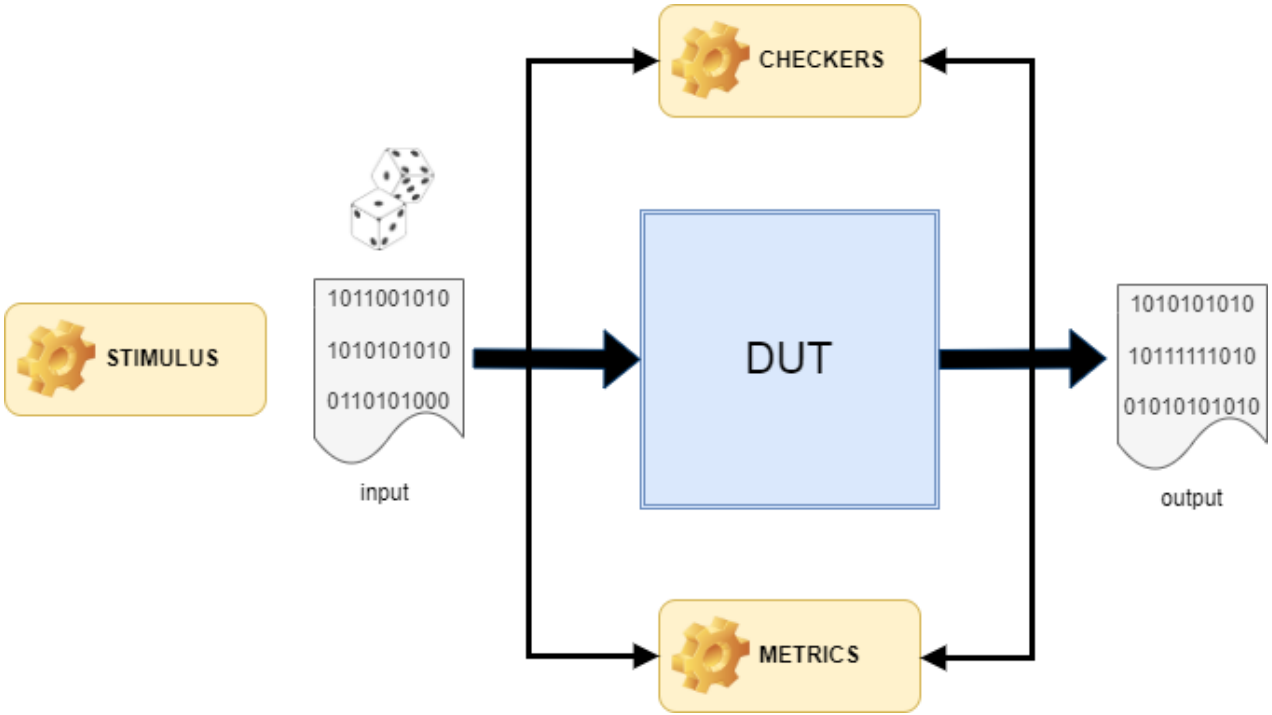
The basics



The basics



CRV


CHECKERS
STIMULUS
1011001010
1010101010
0110101000
input
DUT
1010101010
10111111010
01010101010
output
METRICS

Considerations

> Separation of concerns

> Re-use

> Maintenance

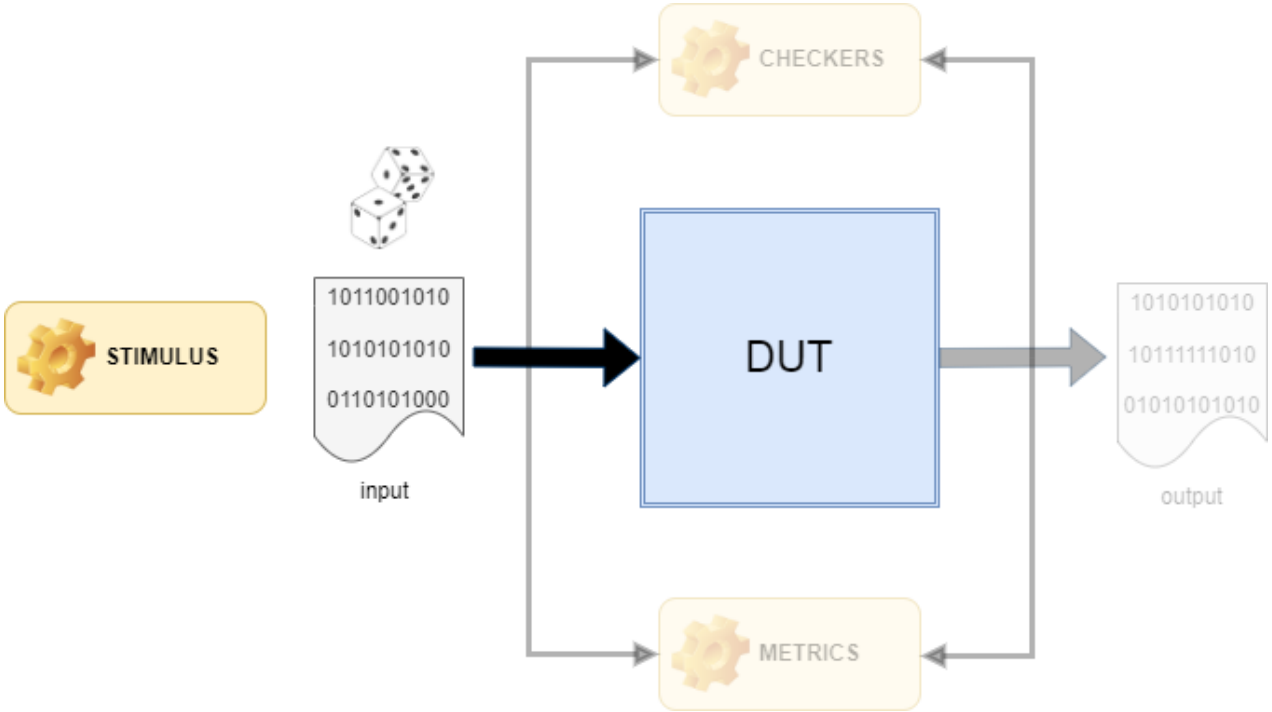
> Verification confidence

> Scaling



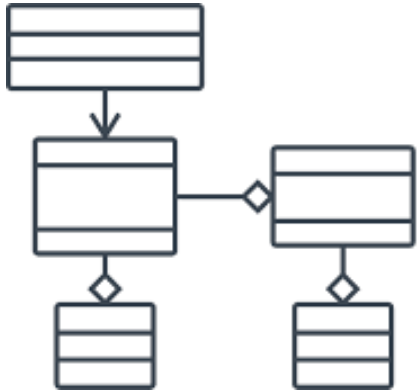
Stimulus

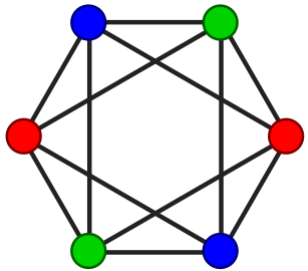
Stimulus


CHECKERS
STIMULUS
1011001010
1010101010
0110101000
input
DUT
1010101010
10111111010
01010101010
output
METRICS

Stimulus

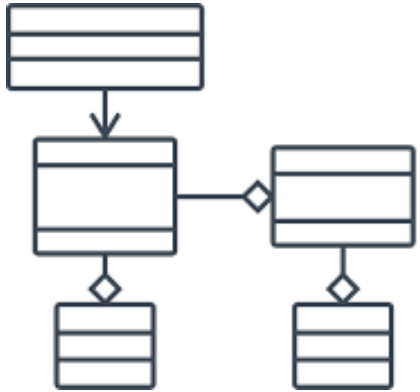
1. How to model the legal stimuli space
   - Use constraint programming.
   - Part of most advanced hardware verification languages, e.g. SystemVerilog
2. Model stimulus at a suitable abstraction level.
3. Control the results, or “load” the dice.

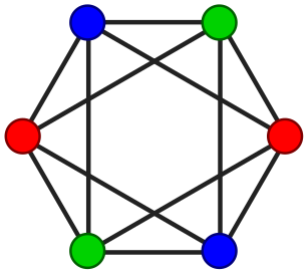




Constrain Programming

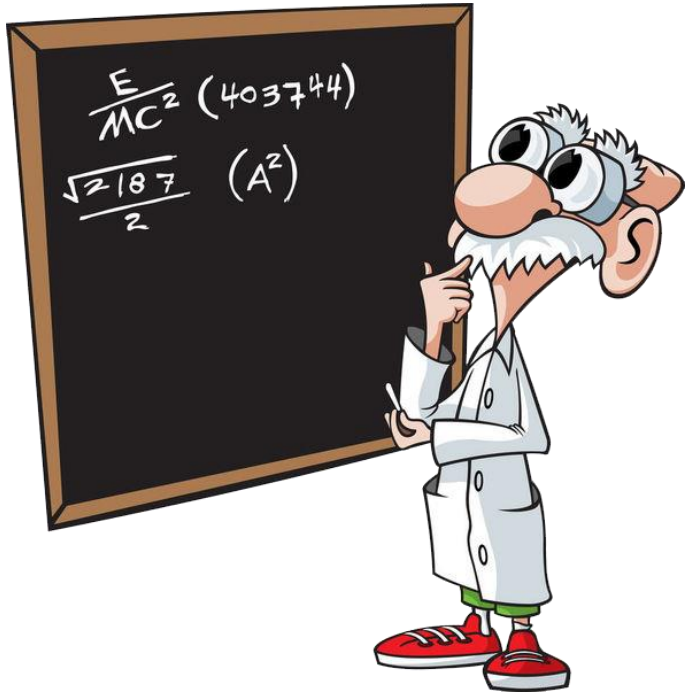
- Declarative
  - Describe the “what”, rather than the “how”.
- Constrains Satisfaction Problem (CSP)
  - Mathematical model relating free variables
  - Search problem (find solution or prove none exist)
- Constraints
  - Boolean, linear, and non-linear relations
- Methods
  - Brute force
  - Backtracking
  - Heuristics
  - …





Constraints in CRV

- Defines legal values
- Compact
  - loops, conditionals, set operators, *etc*.
- Constraint blocks are class members
- Solver find random solutions satisfying the constraints



Constraints Example

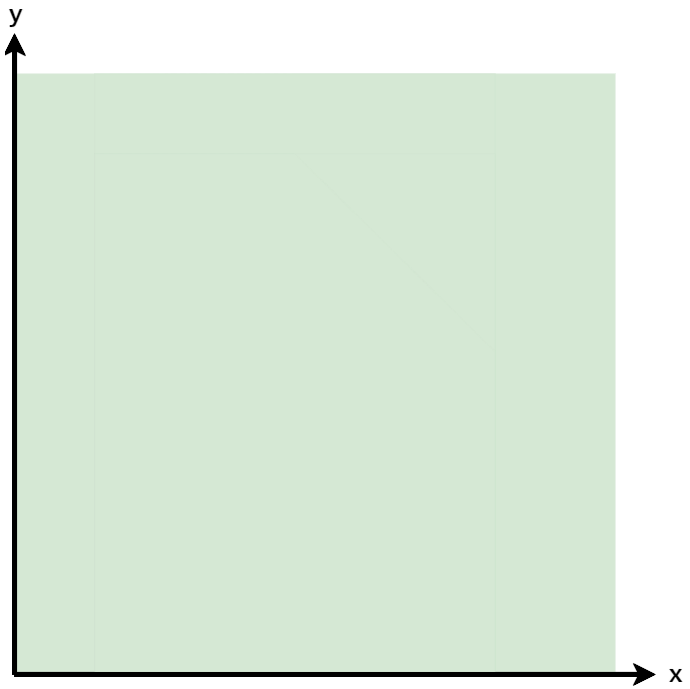
```
rand logic [3:0] x;
rand logic [3:0] y;

constraint c_x { x inside {[2:12]}; }

constraint c_y { y < 14; }

constraint c_sum { x + y < 21; }

constraint c_x_mod { x % 8 != 0; }
```



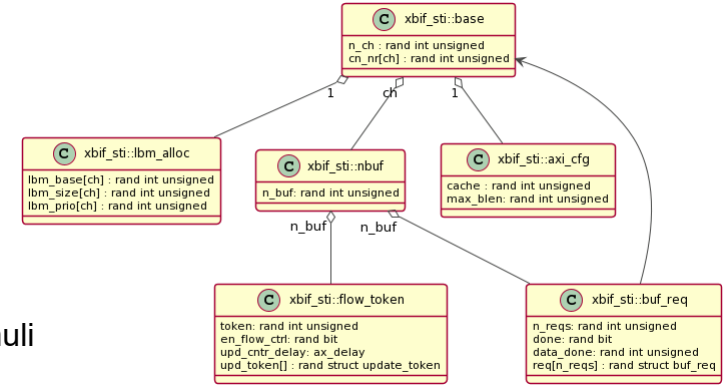
Controlling Random Distribution

- Random distribution is rectangular by default
- In many cases, we need other random distributions to hit all interesting cases
- Define alternative sets of constraints
  - Use subclasses
- Add constraints to limit the solution space
- Add **dist** constraints to control random distribution of a variable



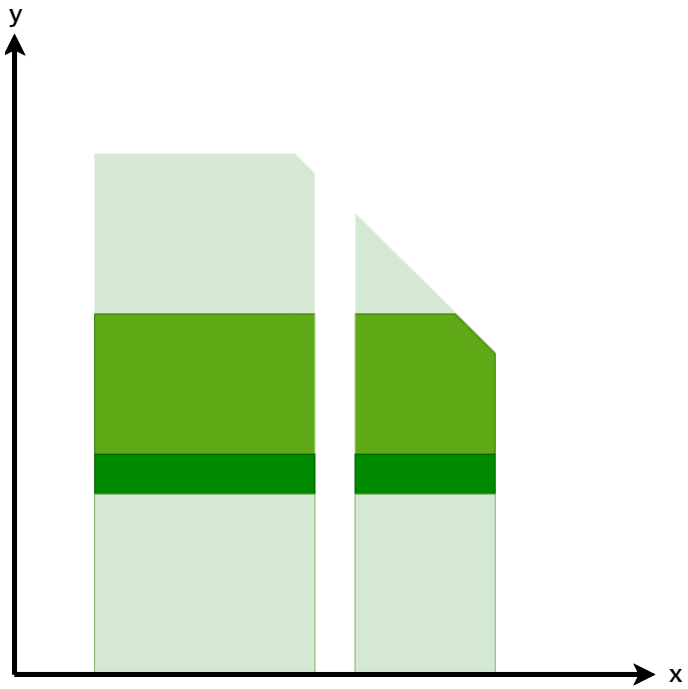
Modeling Stimuli

- Model the input to the DUT as random variables
  - Configuration and data
  - Structure using object-orientation
  - Consider reuse of parts
- Express legal stimuli using constraints
- Abstract low-level stimuli when appropriate
  - To simplify constraints
  - To enable us to randomize interesting cases
  - Randomize high-level “knobs” and map to concrete stimuli
- Appropriate granularity of randomization
  - Randomize at once or sequence the randomizations
  - Tradeoff between expressiveness and performance



Distribution Constraint Example

```
constraint c_d {
  y dist { [0:4] := 1,
             5   := 5,
           [6:9] := 2 }
}
```



Constrain Solver

```
class base;

  rand bit a;
  rand byte unsigned b;

endclass: base
```

```
class derived extends base;

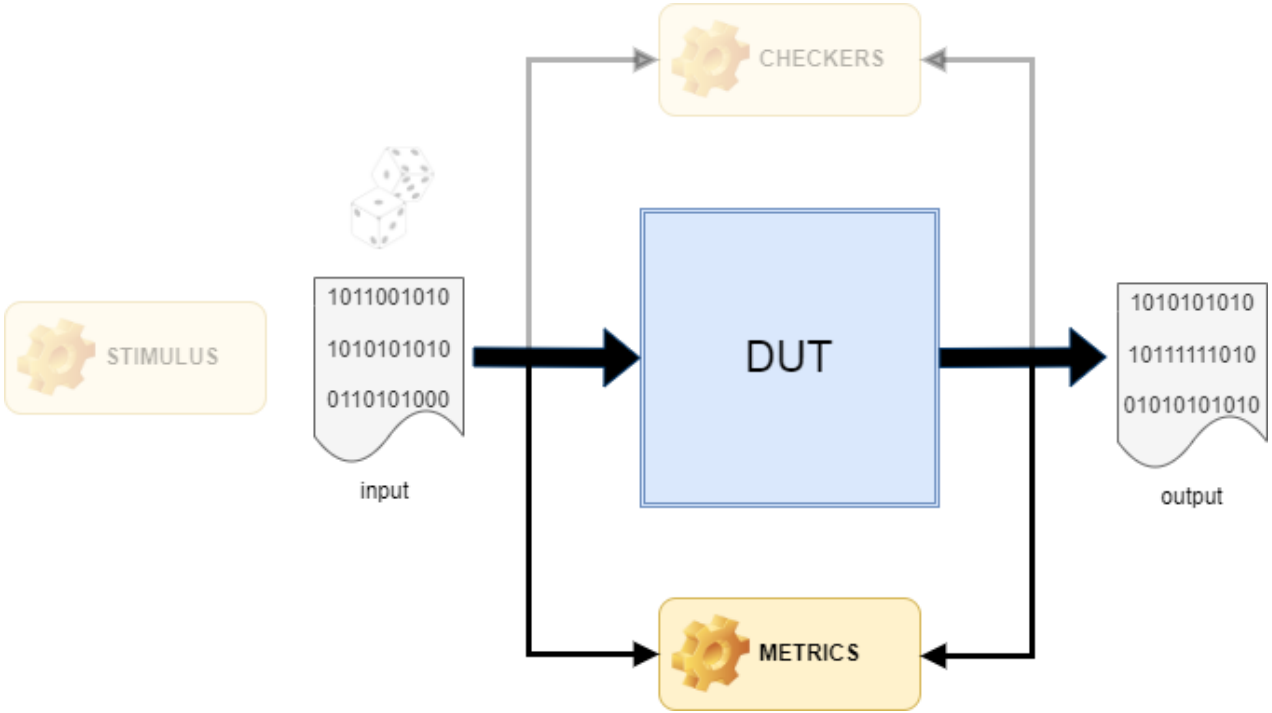
  constraint a_0 {
    (a == 0) -> (b == 0);
  }

endclass: base
```

a	b	base	derived
0	0	1/512	1/257
0	1	1/512	0
0	2	1/512	0
…	…	…	…
0	254	1/512	0
0	255	1/512	0
1	0	1/512	1/257
1	1	1/512	1/257
1	2	1/512	1/257
1	3	1/512	1/257
…	…	…	…
1	254	1/512	1/257
1	255	1/512	1/257

Metrics

Metrics


CHECKERS
STIMULUS
1011001010
1010101010
0110101000
input
DUT
1010101010
10111111010
01010101010
output
METRICS

Coverage

- Primary metric for measuring completeness
- Two main types:
  - Code Coverage
  - Functional Coverage
- Collected during simulation by the simulator
- Analyzed to determine which aspects have not been fully verified yet



Controllability and observability

> Controllability: ability to influence or activate an internal part of the design by putting some stimuli on the input ports

  - Good controllability on input ports themselves, but what about deep internal states?



> Observability: Ability to observe the effects of some internal state of the design

  - Only observing the output ports may not give very good observability!

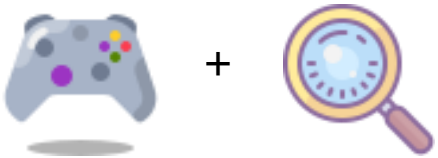


Icons from icons8.com

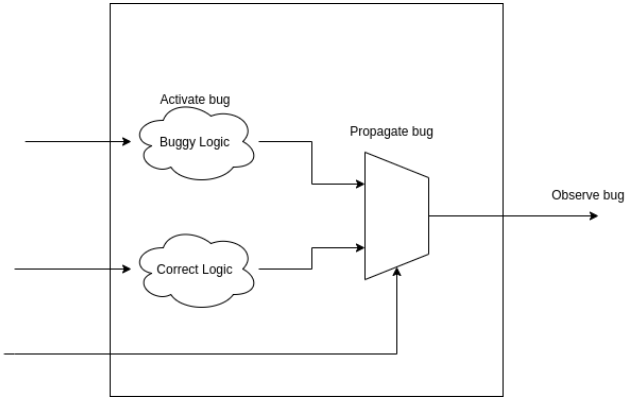
Controllability and observability

> To detect a bug the test bench must:
  - Generate input stimuli that activates the bug
  - Generate input stimuli to propagate the bug to an observable point
  - Monitor the observable point with some sort of checking to detect the bug

> Must have both controllability and observability to find bugs!



Icons from icons8.com



What is coverage?

> Coverage is a metric that measures the **controllability** of the test bench

> Helps us know if the verification is **complete**

> Tells us what parts of the functionality have been exercised during simulation

> Tells us what parts of the functionality **have not** been exercised during simulation

Types of coverage

Method of creation → Source	Implicit	Explicit
Implementation	Code coverage	Assertions
Specification	TBD	Functional coverage

Types of coverage

Method of creation → Source	Implicit	Explicit
Implementation	Code coverage	Assertions
Specification	TBD	Functional coverage

Code Coverage

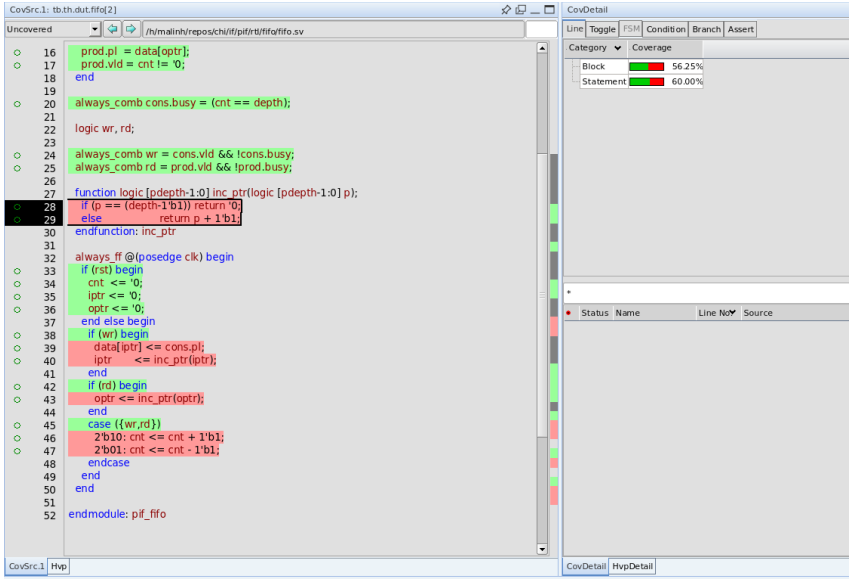
- Implicit
- Based on the code
- Different types:
  - Line Coverage
  - Branch Coverage
  - Toggle Coverage
  - Condition Coverage
  - FSM Coverage



Code coverage - Line

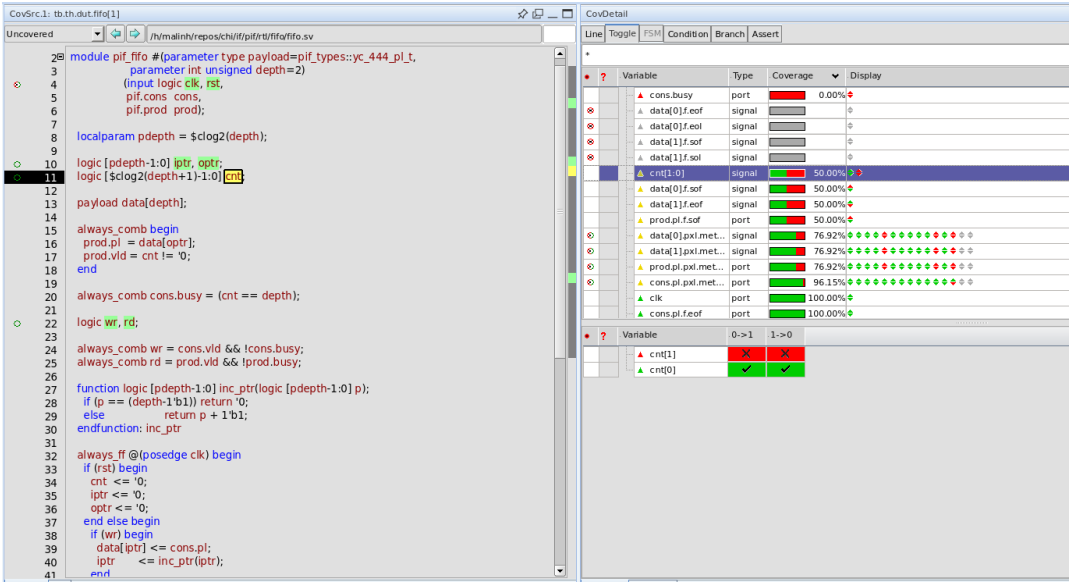
> Which lines have been executed

> (optionally how many times each line has been executed)



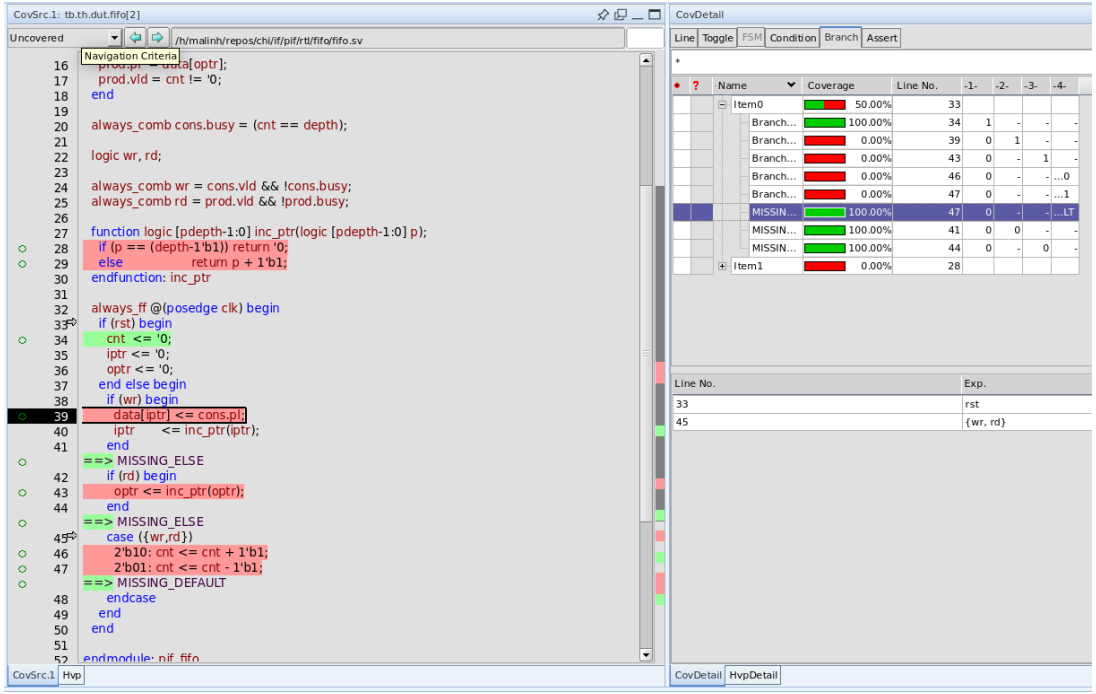
Code coverage - Toggle

- Each bit in RTL has toggled 0→1 and 1→0
- **A lot** of coverage points from this!
- Formal tools for excluding unreachable points



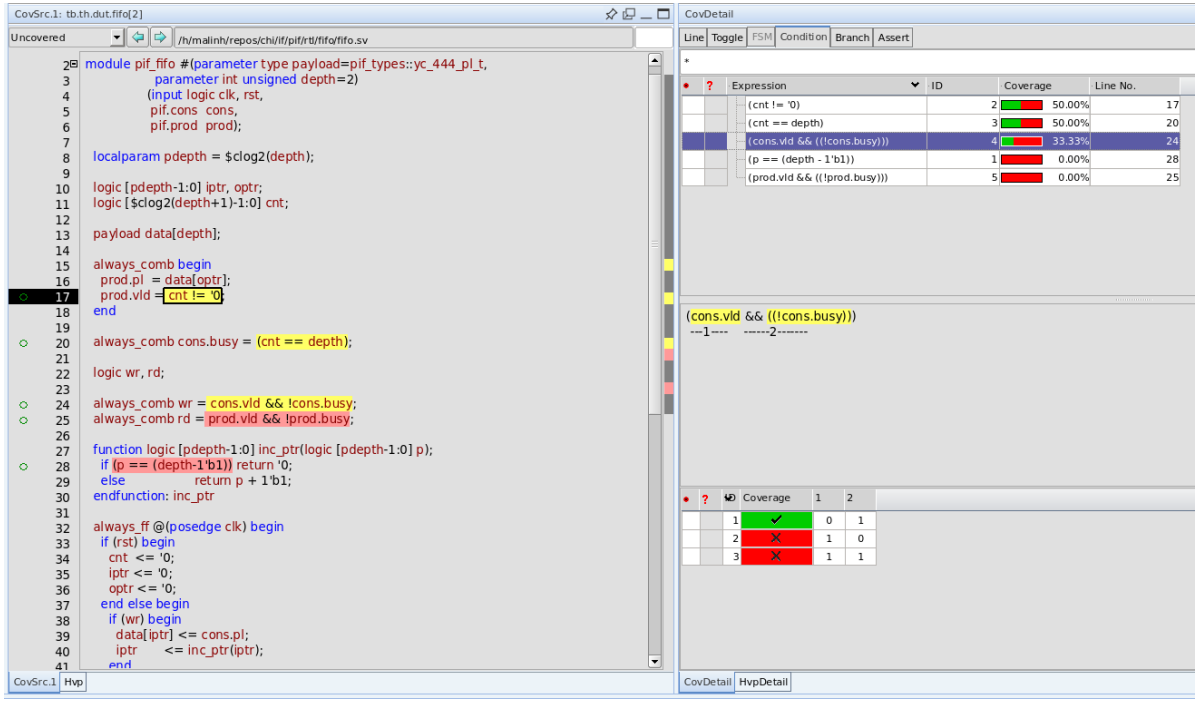
Code coverage - Branch

- Each possible branch taken of control structure like if, case
- Similar to line coverage
  - Also covers missing else/default



Code coverage - Condition

- Each condition evaluate to true and false
- Parts of larger expressions checked



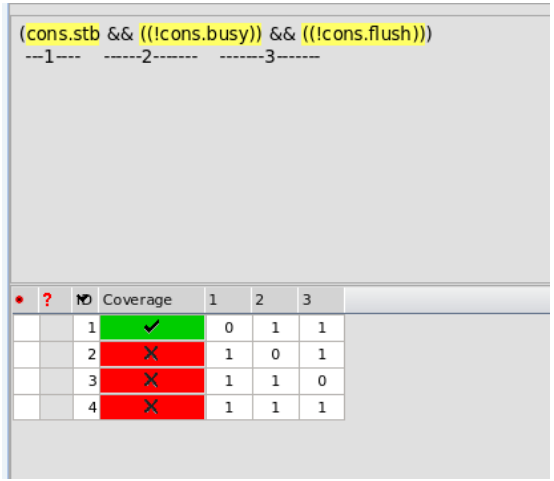
Code coverage – Condition vs branch

> Branch coverage: branch taken yes or no

```
58    if (cons.stb && !cons.busy && !cons.flush) begin
59      nxt_data = nxt_data | ((cons.data & mask) << (nxt_cnt*GRAN));
60      nxt_cnt
61      wr_data
62    end
==> MISSING_ELSE
63  end
```

59 Branch2.0 Uncovered
Press 'F3' to save in Tooltip Viewer

> Condition coverage: all combinations for taking branch tested



Code coverage – Finite State Machine

> Identify FSM in RTL

> # of times each state was entered

> Transitions between states

	s0	s1	s2	s3	s4	s5
s0	-		-	-		
s1	-	-				
s2			-			
s3		-		-		
s4				-	-	
s5			-	-	-	-

Typical code coverage flow

1) Activate collection of code coverage

2) Run simulations to collect code coverage

3) Report and analyze results

4) → Analyze the reason for missing coverage, “coverage holes”

Analyze coverage holes

Problem

> Missing input stimuli to activate code (Controllability)

> Bug in the design (or testbench)

> “Known” unused or dead code, unreachable during normal operating conditions



Solution



Analyze coverage holes

Problem

> Missing input stimuli to activate code (Controllability)

> Bug in the design (or testbench)

> “Known” unused or dead code, unreachable during normal operating conditions



Solution

> Update stimuli: constraints, distributions, new sequences/test cases



Analyze coverage holes

Problem

> Missing input stimuli to activate code (Controllability)

> Bug in the design (or testbench)

> “Known” unused or dead code, unreachable during normal operating conditions



Solution

> Update stimuli: constraints, distributions, new sequences/test cases

> Fix the bug!



Analyze coverage holes

Problem

> Missing input stimuli to activate code (Controllability)

> Bug in the design (or testbench)

> “Known” unused or dead code, unreachable during normal operating conditions



Solution

> Update stimuli: constraints, distributions, new sequences/test cases

> Fix the bug!

> Remove if possible, or instruct the coverage tool to exclude this code when collecting and reporting coverage



Code coverage summary

Pros

> Extracted automatically from RTL, no effort required

> Identify parts of the code that have not been exercised

> Simple way to identify stimuli deficiencies

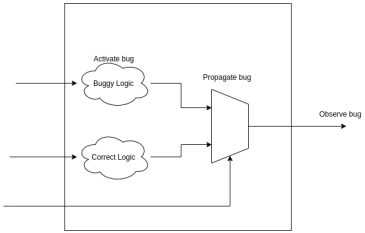
Cons

> Not enough on it’s own

> No coverage generated for functionality not implemented

> No way to tell the 3 conditions for finding a bug has been fulfilled at the same time

- Activate bug
- Propagate bug
- Observe bug



> Does not capture use intentions

Types of coverage

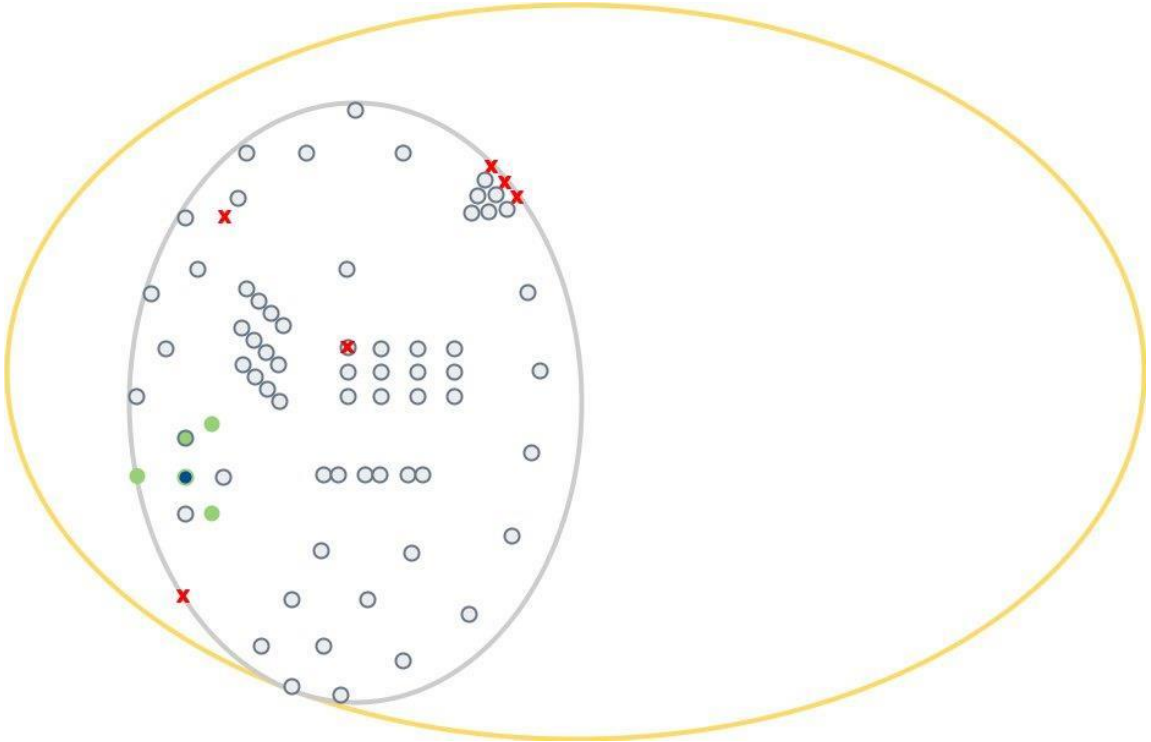
Method of creation → Source	Implicit	Explicit
Implementation	Code coverage	Assertions
Specification	TBD	Functional coverage

Functional Coverage

- Explicit
- Based on specification
- Defines interesting cases
- Sample values during simulation:
  - Configuration
  - Input data
  - Output data
  - Internal state
  - Combinations of the above
- Use knowledge about the design
- Consider observability



Functional coverage



Functional coverage

- Defining coverage inputs
  - Specification and description
  - Use cases
  - Corner cases
  - Design engineer
  - Discontinuities
  - Risky areas in RTL
  - Other verification engineers



Modeling functional coverage

> Cover group

> Cover property

Modeling functional coverage - covergroups

- Captured at a specific point in time
- Values of variables, interface signals, registers, inputs, outputs
- Typically implemented as covergroups in SystemVerilog
- Example
  - Value of data on an interface at the point of transfer.
  - Value of all configuration registers at the start of processing
  - Value of status registers at the end of processing

Functional Coverage Example

```
covergroup cg_upsamp_cfg;

  cp_subsamp: coverpoint subsamp {
    bins jpeg       = {k_jpeg};
    bins h26x       = {k_h26x};
    bins nnv        = {k_nnv};
    illegal_bins ib = default;
  }

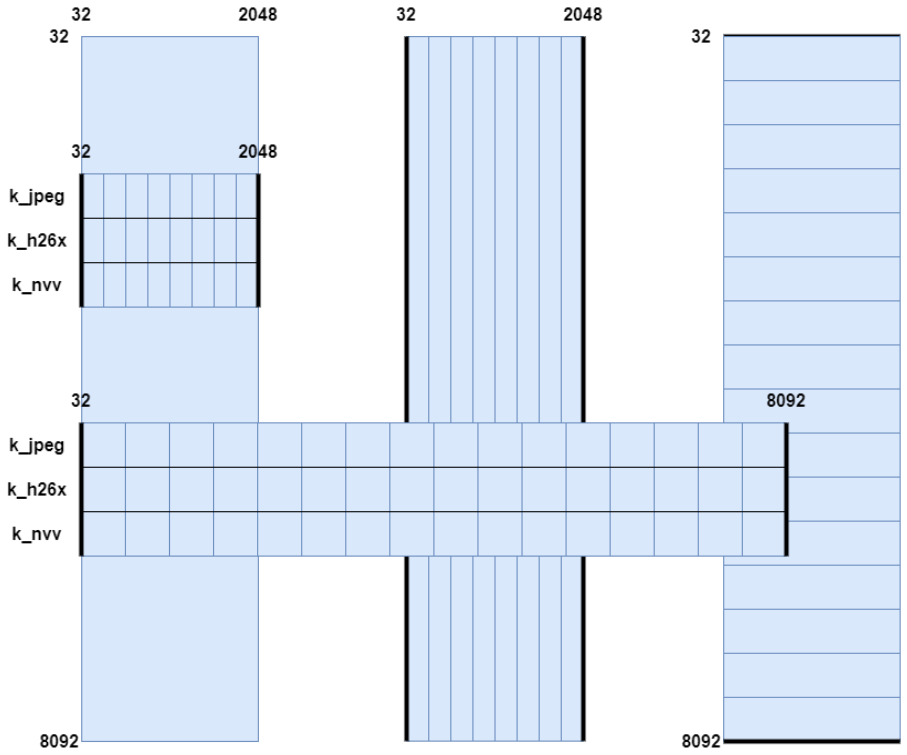
  cp_xs: coverpoint xs {
    bins min          = {       32};
    bins max          = {     2048};
    bins xsize[ 8]    = {[32:2048]};
    illegal_bins bad = default;
  }

  cp_ys: coverpoint ys {
    bins min          = {       32};
    bins max          = {     8192};
    bins ysize[16]    = {[32:8192]};
    illegal_bins bad = default;
  }

  cc_ss_xs : cross cp_subsamp, cp_xs;

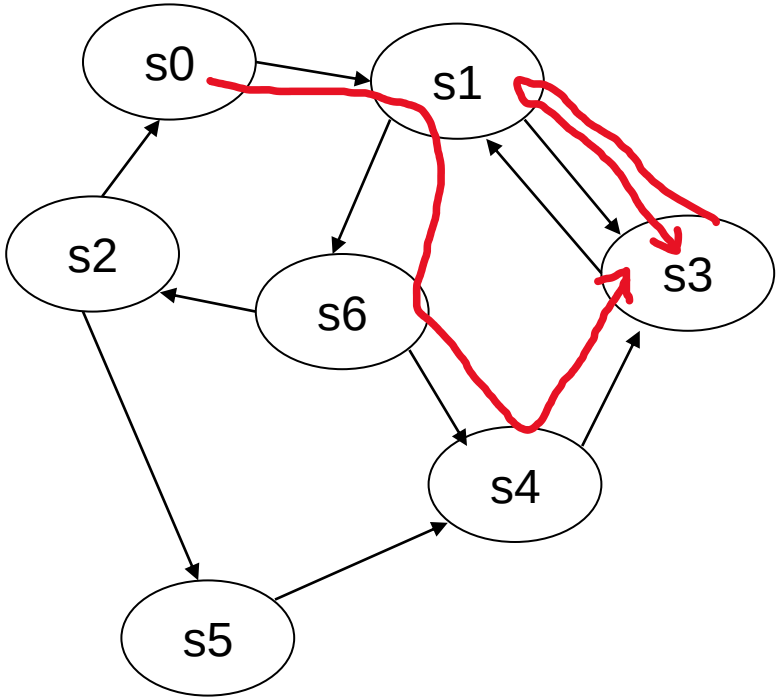

  cc_ss_ys : cross cp_subsamp, cp_ys;

endgroup: cg_upsamp_cfg;
```

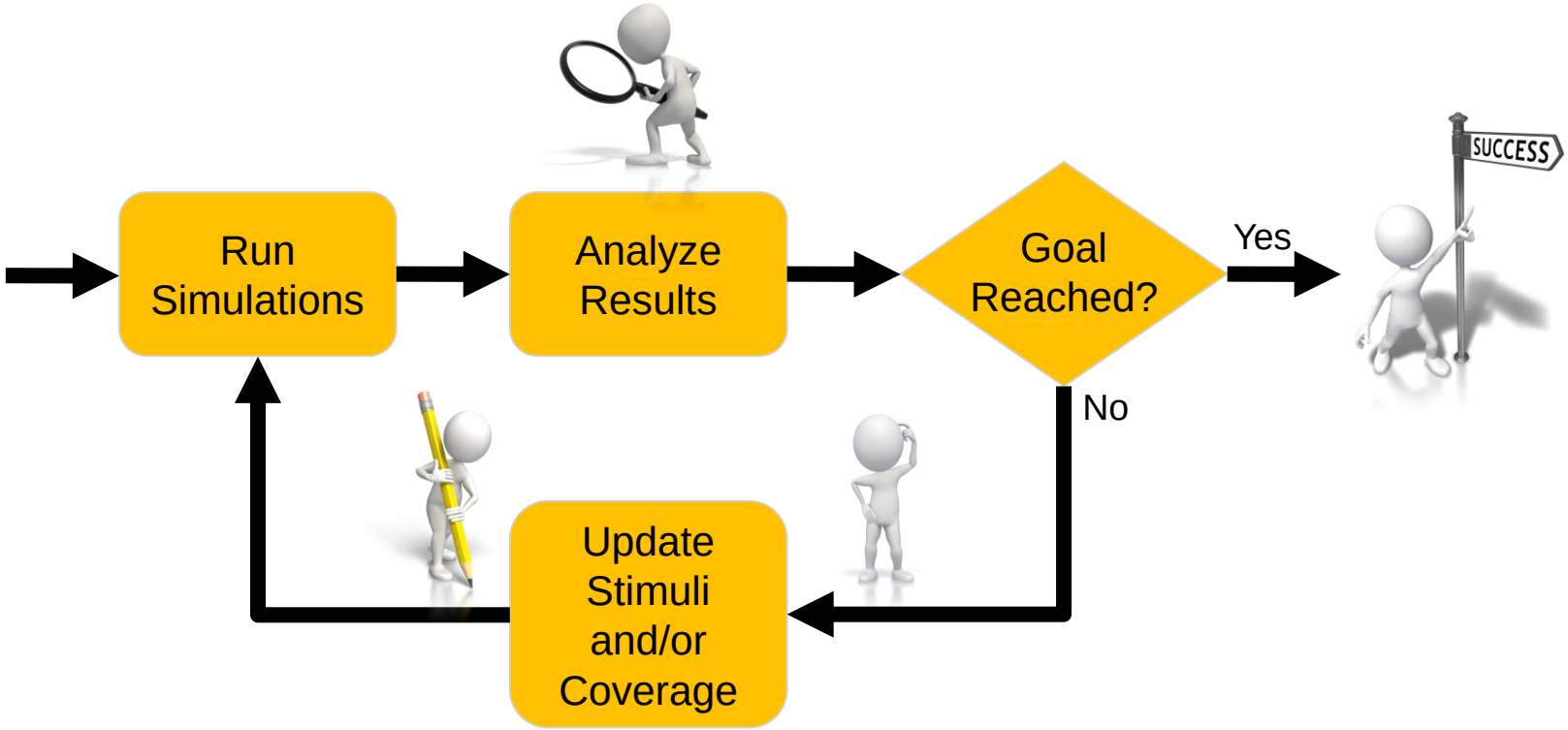


Modeling functional coverage – cover properties

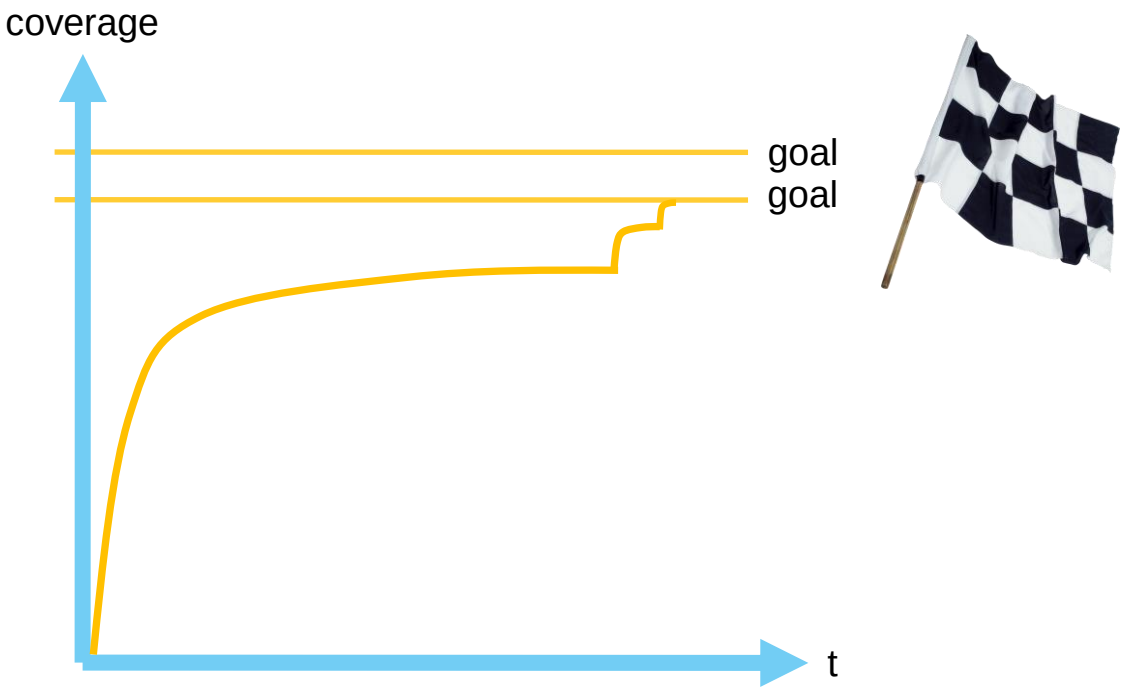
- Captures temporal relationships
- Sequences of events
- Example:
  - Handshake sequence on a bus protocol
  - A specific path through a state machine
  - Sequence of transfers on a bus, such as write then read
- Can be implemented with SVA



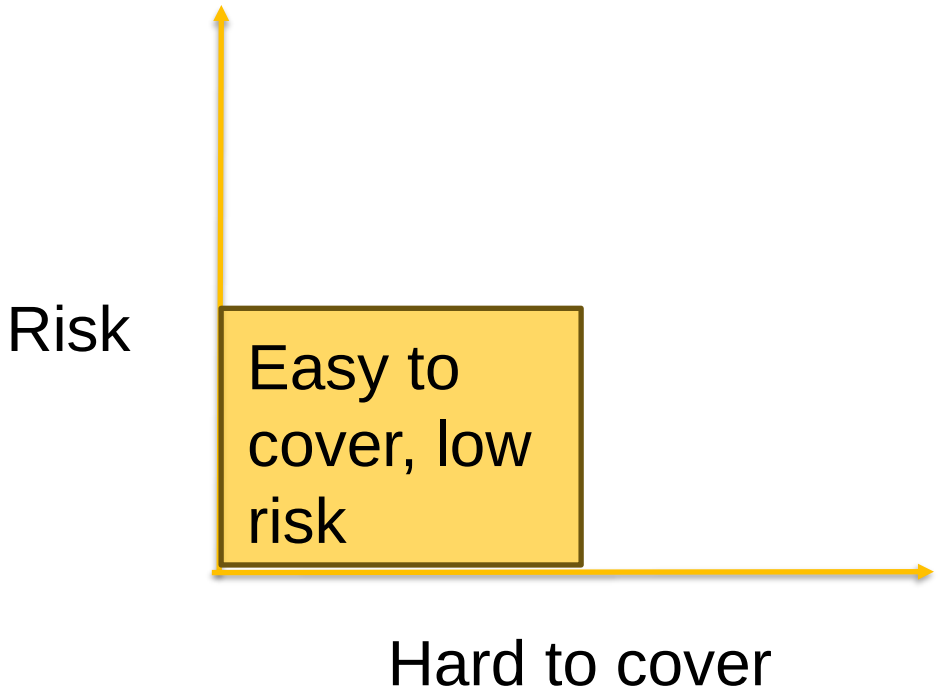
Closing Coverage



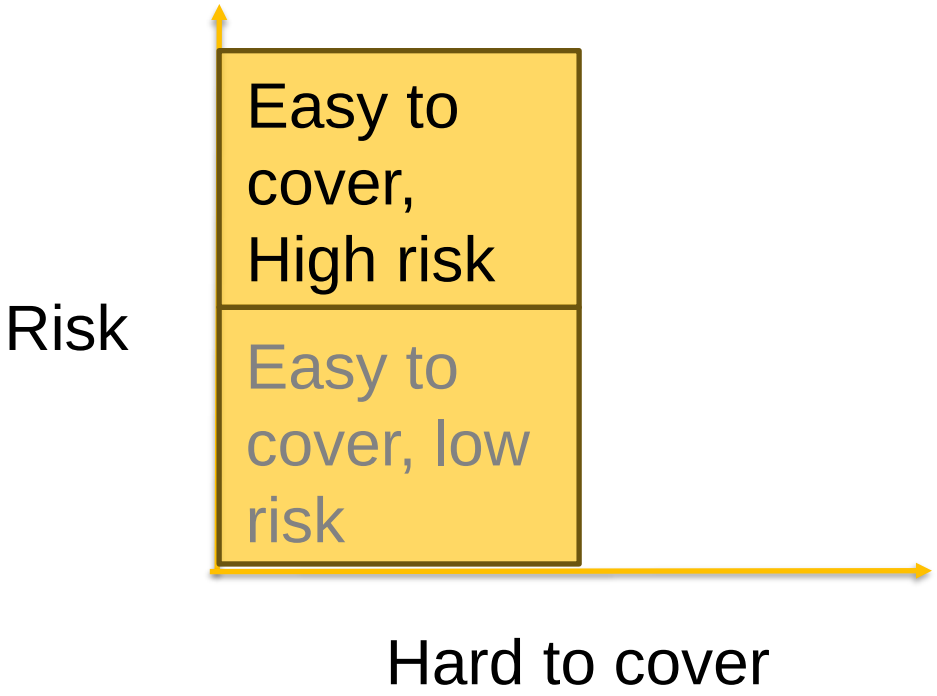
Coverage Closure



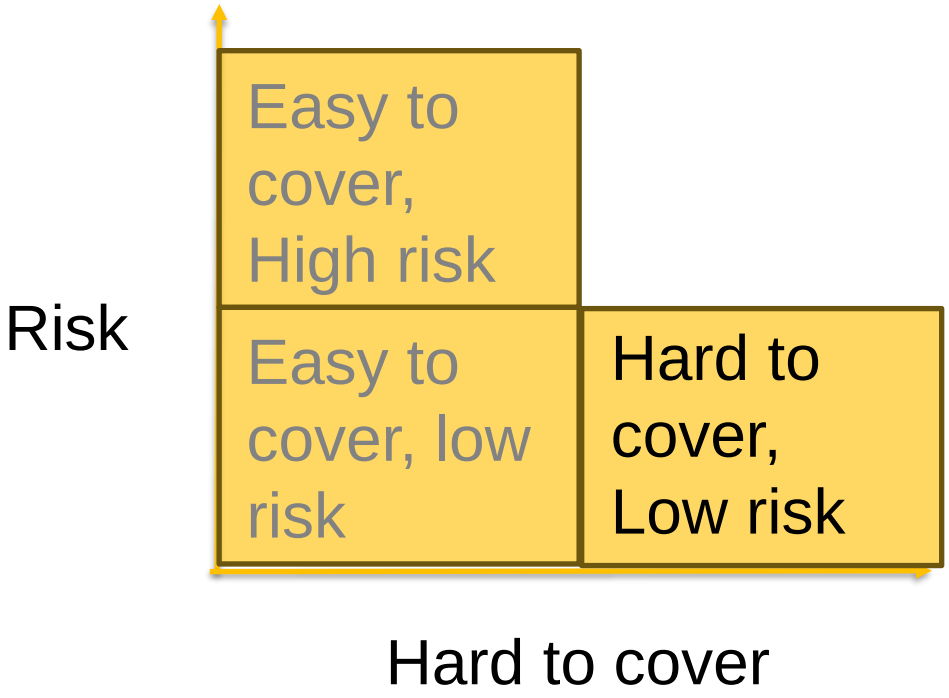
Coverage closure – moving the target



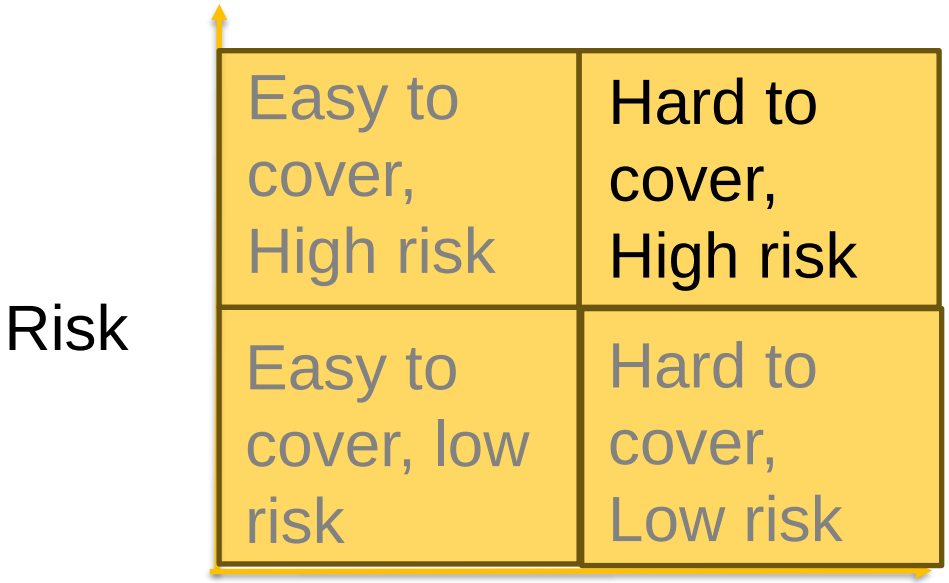
Coverage closure – moving the target



Coverage closure – moving the target

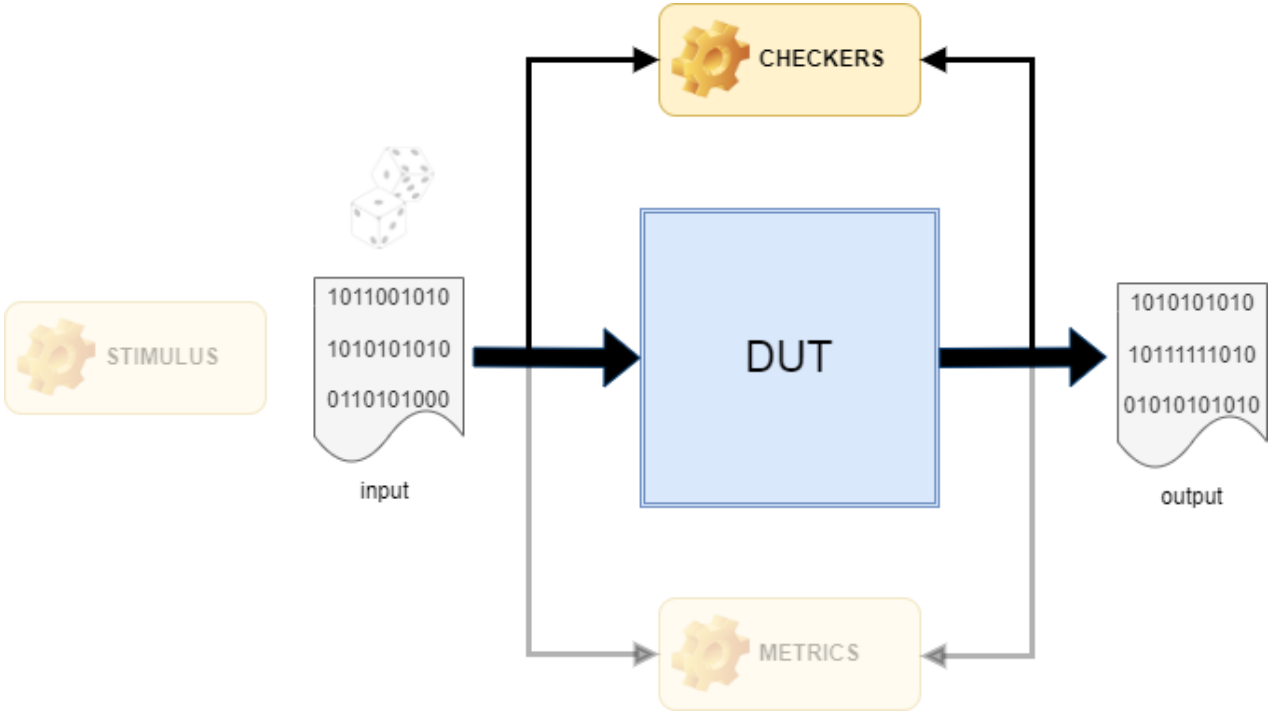


Coverage closure – moving the target



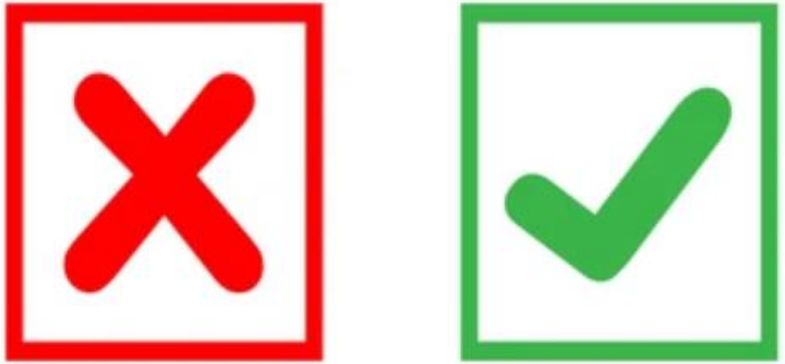
Checkers

Checkers


CHECKERS
STIMULUS
1011001010
1010101010
0110101000
input
DUT
1010101010
10111111010
01010101010
output
METRICS

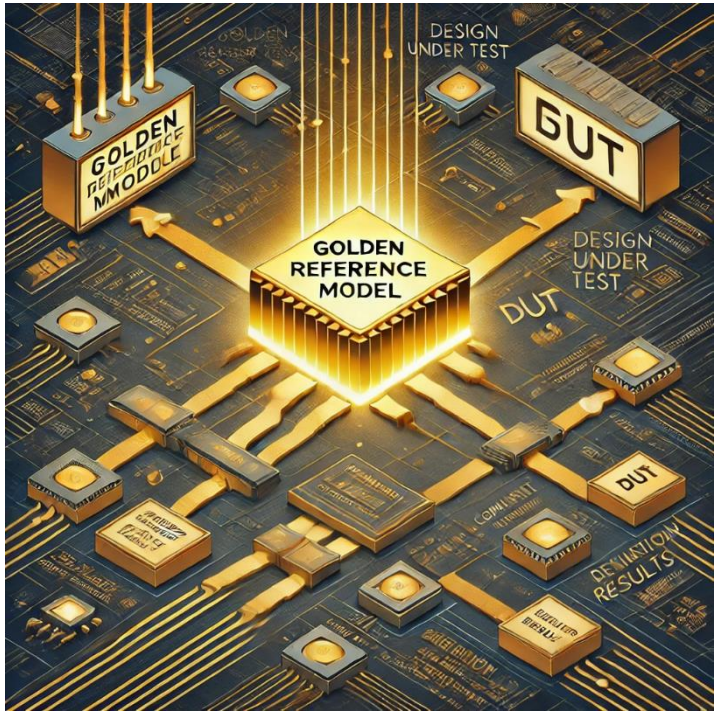
Correctness

> Mechanism to check correct DUT behavior
  - Self-checking
  - Scoreboard
  - Partitioning



Self-checking

- Reference model
  - Abstraction level
    - Transaction
    - Cycle exact
  - Accurateness
    - Bit-exact
    - …
  - Language
    - Python
    - C/C++
    - HDL
    - Assertions



Assertions

- Procedural programming
- `assert` statement used as early as the 1960s in Algol W
- Useful
  - Debugging
  - Documentation

```
def div(a, b):
     assert b!=0
     return a/b
```

Assertions in HDLs

- Introduced in the 90s in HDLs
  - VHDL 93
  - Property Specification Language (PSL) in 2000s
  - SystemVerilog Assertions (SVA) in 2000s
- Procedural
  - Procedural statements
  - Equivalent to assertions in general purpose programming languages.
- Concurrent
  - Concurrent statements
  - Under control of a clock
  - Much more …



Concurrent assertions

- Declarative language for modelling a system
- Belongs to a set of languages called temporal logic
- Usually used to model temporal behaviour, i.e. behaviour over time
- Layered
  - Boolean
  - Temporal
  - Directives



Layers

- Properties
  - Boolean-valued expressions, built from variables and operators
  - Temporal relationships between the boolean expressions, if any. Referred generally as **sequences**

```
property even_value;
  (value%2==0);
endproperty
```

- Directive
  - Instruction to a verification tool that it should attempt an action on an expression, e.g., **assert**, **cover** or **assume** an expression.

```
assert property ( even_value );
```

Temporal behaviour

- Implication
  - Conditional evaluation of *consequent* based on the outcome of an *antecedent* expression.
  - Implication operators
    - overlapping
    - non-overlapping

```
property pulse;
  (a |-> #1 !a);
endproperty
```

```
antecedent OPERATOR consequent
```

Temporal behaviour

> Cycle delays

- Models the number of cycles between boolean expressions.
- In SystemVerilog denoted by `##N` where N is a positive integer
- Use of regular expressions and syntactic sugaring to make concise and precise properties
  - Concatenation
  - Repetition
  - Ranges
  - Bounding
  - …

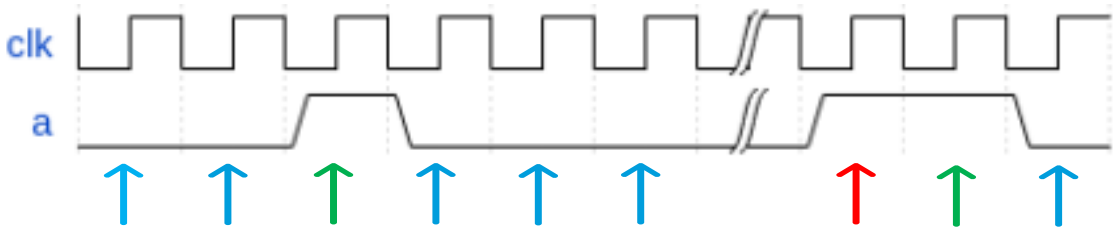
```
property long_pulse;
  (!a ##1 a |-> ##1 a ##1 !a);
endproperty
```

```
property long_pulse;
  ($rose(a) |-> a[*2] ##1 !a);
endproperty
```

Temporal behaviour

> Properties are checked on every clock cycle

```
property pulse;
   (a |-> #1 !a);
endproperty
```



Assertion Based Verification (ABV)

- Derive properties from specification
- Abstract representation of the DUT
- Use assert directive on properties to check the correctness of the design
  - Success
  - Vacuous success
  - Fail



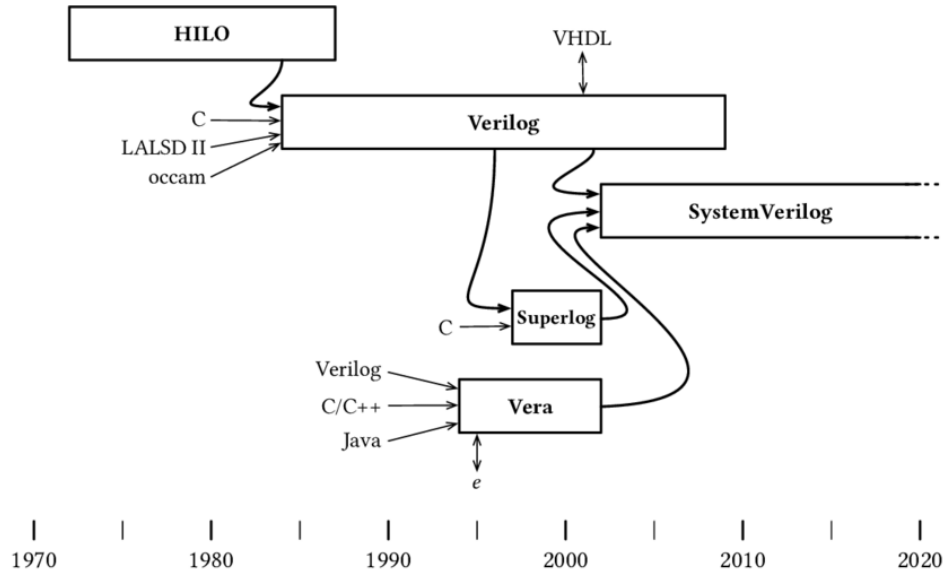
SystemVerilog

SystemVerilog

- Design Features
  - Synthesizable
  - Extends on Verilog
- Verification Features
  - Classes
  - Constraints
  - Coverage
  - Assertions
- Developed by the Accellera System Initiative
- Std IEEE 1800-2023: https://ieeexplore.ieee.org/document/10458102



SystemVerilog – family tree



Flake, Peter & Moorby, Phil & Golson, Steve & Salz, Arturo & Davidmann, Simon. (2020). Verilog HDL and its ancestors and descendants. Proceedings of the ACM on Programming Languages. 4. 1-90. 10.1145/3386337.

SystemVerilog





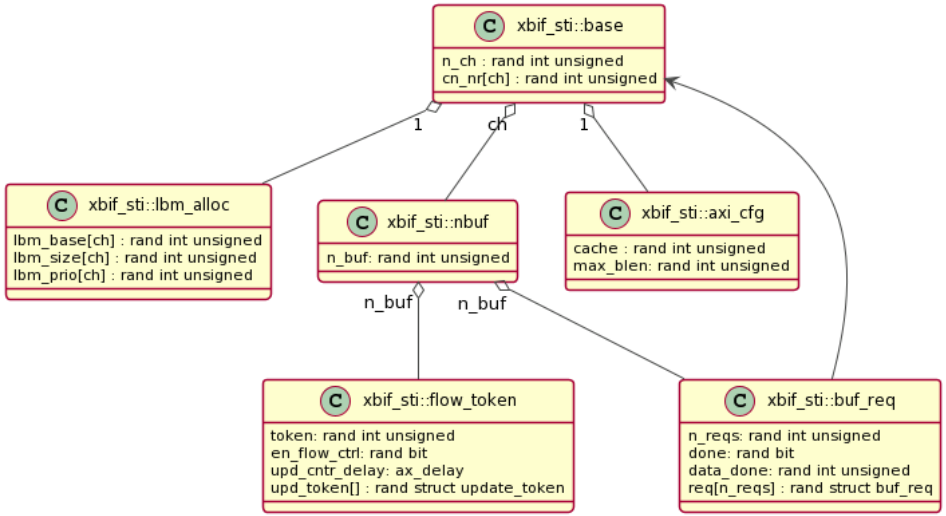
Code Reuse

- Test benches are complex software systems
- Large amount of code
- Many test benches for a chip
- Code reuse is essential
  - Across similar test benches – Horizontal
  - Across hierarchy levels – Vertical
  - Across iterations of the chip



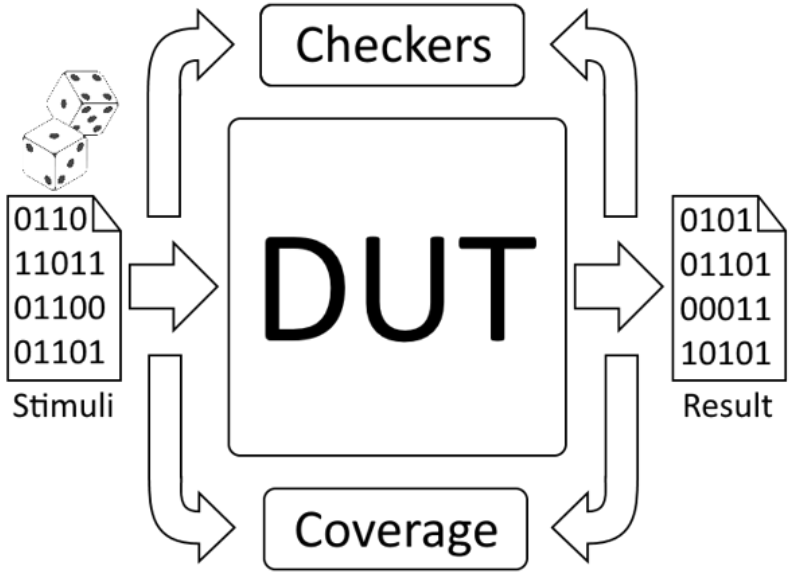
Object oriented programming

- Classes
- Inheritance
- Polymorphism
- Encapsulation
- Provides flexibility and opportunity for reuse!



Communication with other languages

- DPI – Direct Programming Interface
- Provides an interface between SV and other language
- Call C functions from SystemVerilog
- Export SystemVerilog functions to C



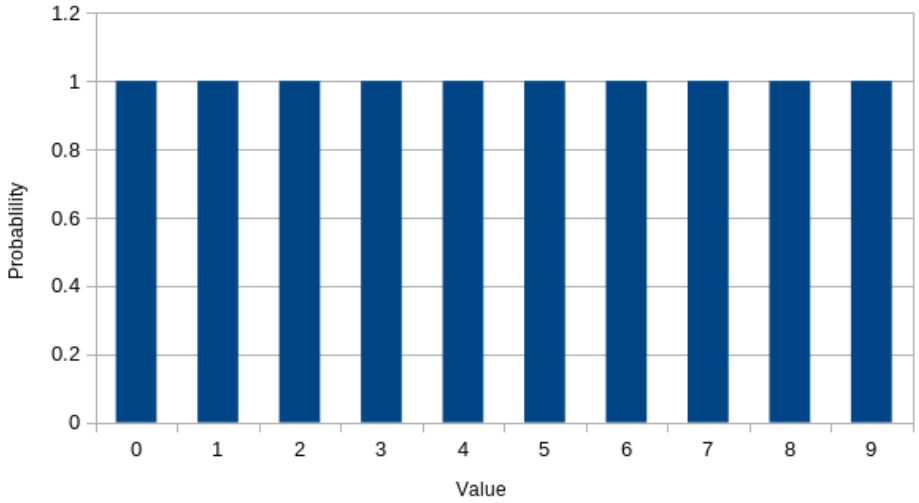
Constraints and randomization - constraints

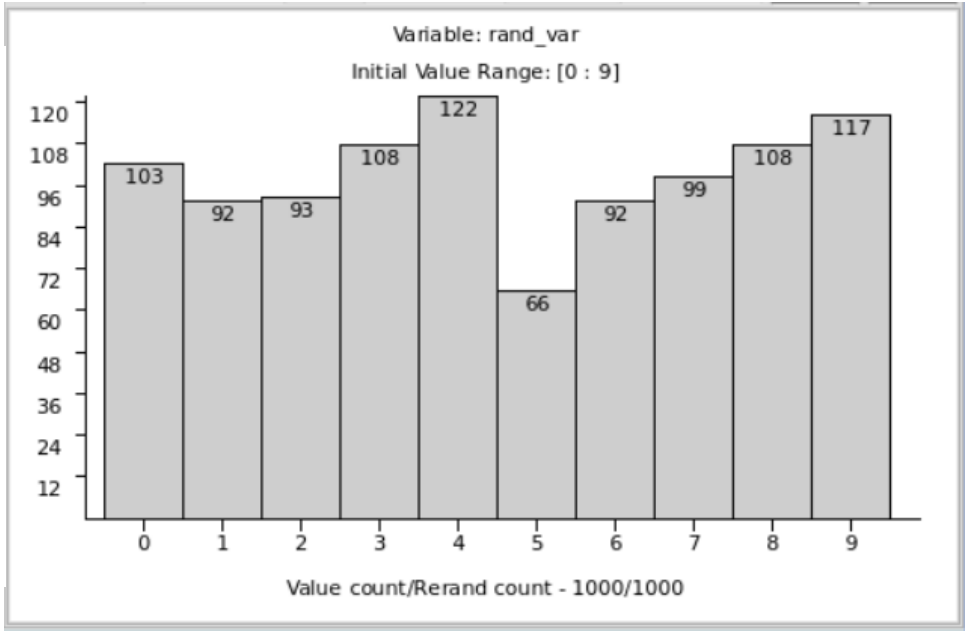
> Sets up the rules of the **randomizable variables**
  - Describes the problem
  - Constraint solver will provide a solution!
> **Rand** keyword declares variables randomizable



Constraints and randomization - constraints

> **Rand** int n; 0-9





Constraints in SystemVerilog

> https://www.chipverify.com/systemverilog/systemverilog-constraints

> Constraints are bidirectional!

Constraints in SystemVerilog

> https://www.chipverify.com/systemverilog/systemverilog-constraints

> Constraints are bidirectional!

```
constraint c_smaller {
   a < b;
}
```

Constraints in SystemVerilog

> https://www.chipverify.com/systemverilog/systemverilog-constraints

> Cons

```
constraint c_conditional {
    if(c) {
        a == 5;
    } else {
        a == 3;
    }
}
```

Constraints in SystemVerilog

> Create class

> Call to randomize rand variables

```
class test;
  rand int unsigned a, b;
  rand bit c;

  // Constraints
endclass : test

// ....
test test_inst = new();
test_inst.randomize();
```

Constraints in SystemVerilog

- .randomize()
- $urandom()
- $random()
- $urandom_range(max,min)

- Only works with integral types, not real!
  - Needs to be manually scaled

```
class test;
  rand int unsigned a, b;
  rand bit c;

  // Constraints
endclass : test

// ....
test test_inst = new();
test_inst.randomize();
```

Covergroups

- Implements the modeling of cover groups for functional coverage
- Crosses – 2d (or more) coverage

```
bit [9:0] v_a;

covergroup cg @(posedge clk);

   coverpoint v_a
   {
      bins a = { [0:63],65 };
      bins b[] = { [127:150],[148:191] };    // note overlapping values
      bins c[] = { 200,201,202 };
      bins d = { [1000:$] };
      bins others[] = default;
   }
endgroup
```

Dynamic data types

- Data structures that can't be synthesized but are helpful when writing testbenches
- Queue
- Dynamic array
- Associative array
- Dynamic strings

Properties and assertions

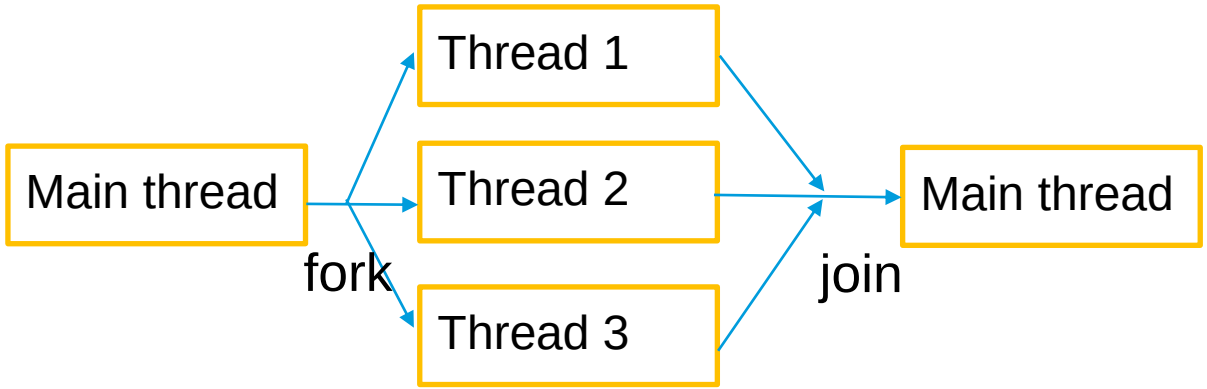
> Modeling of cover properties

> Properties can be **checked** (asserted to be true always or at some specific point) and **covered**.

> Describes temporal behavior

Threading

- Support for parallel execution
  - Fork
  - Join,
    join_any,
    join_none
  - Wait fork
  - Disable fork
- https://verificationguide.com/systemverilog/systemverilog-fork-join/



Class libraries

> UVM – Universal Verification Methodology

- Next lecture!

Case study

Image processing block

Edge detection for Image recognition

Case study – Image size

- Support different pixel resolutions
  - Large / Small
  - Wide / Tall
- Configuration
  - X Size
  - Y Size
  - 16x16 -> 4096x4096









Case study – Cropping

> Possible to crop an area of the image

> Configuration

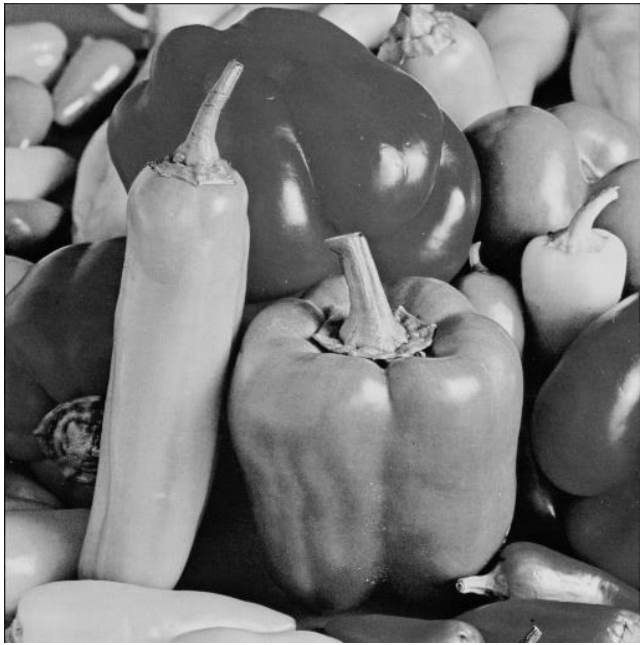
- Two points to represent the cropping.
  - Top left (x0, y0)
  - Bottom right (x1, y1)
  - Within the image



Case study – Grayscale

- RGB to Grayscale conversion
  - Color to Luminance
- Y = 0.2126R + 0.7152G + 0.0722B

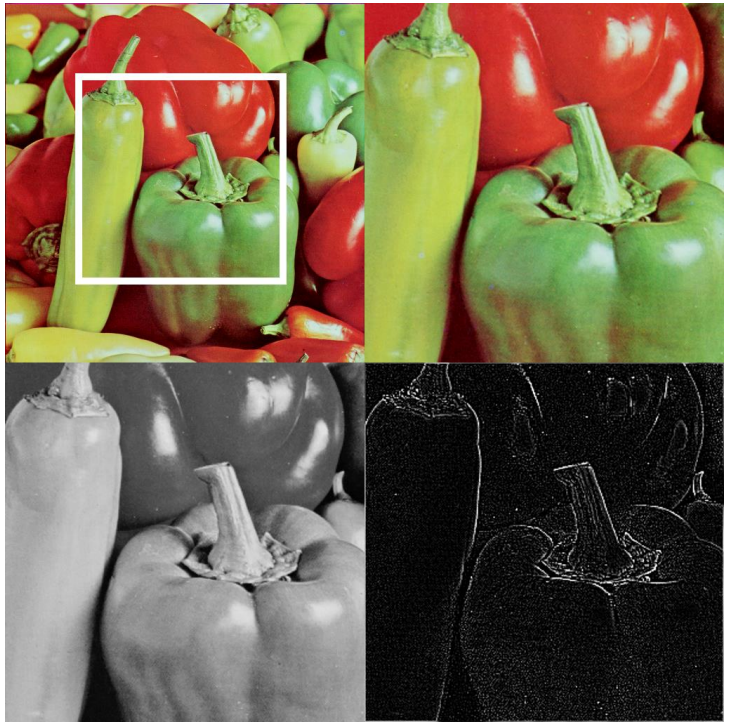
- Configuration
  - Coefficients
    - C_r | C_g | C_b
  - The sum of the coefficients should be 1



Case study – Specification summary

- Size
  - X
  - Y
- Cropping
  - Top left
  - Bottom right
- Grayscale
  - One coefficient for each color component
- Edge detection
  - Enable / Disable edge detection

- $\sim 10^{34}$ combinations of configurations
  - Excluding pixel values

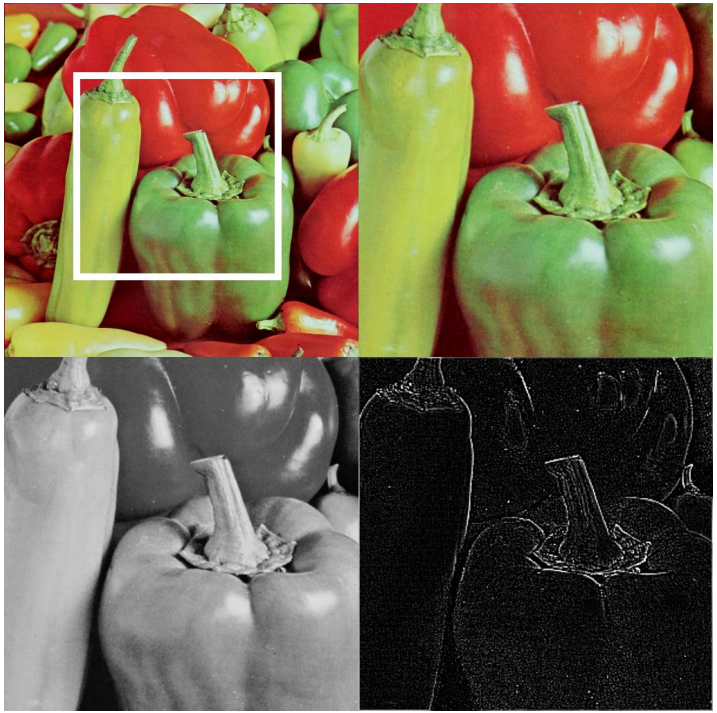


Case Study

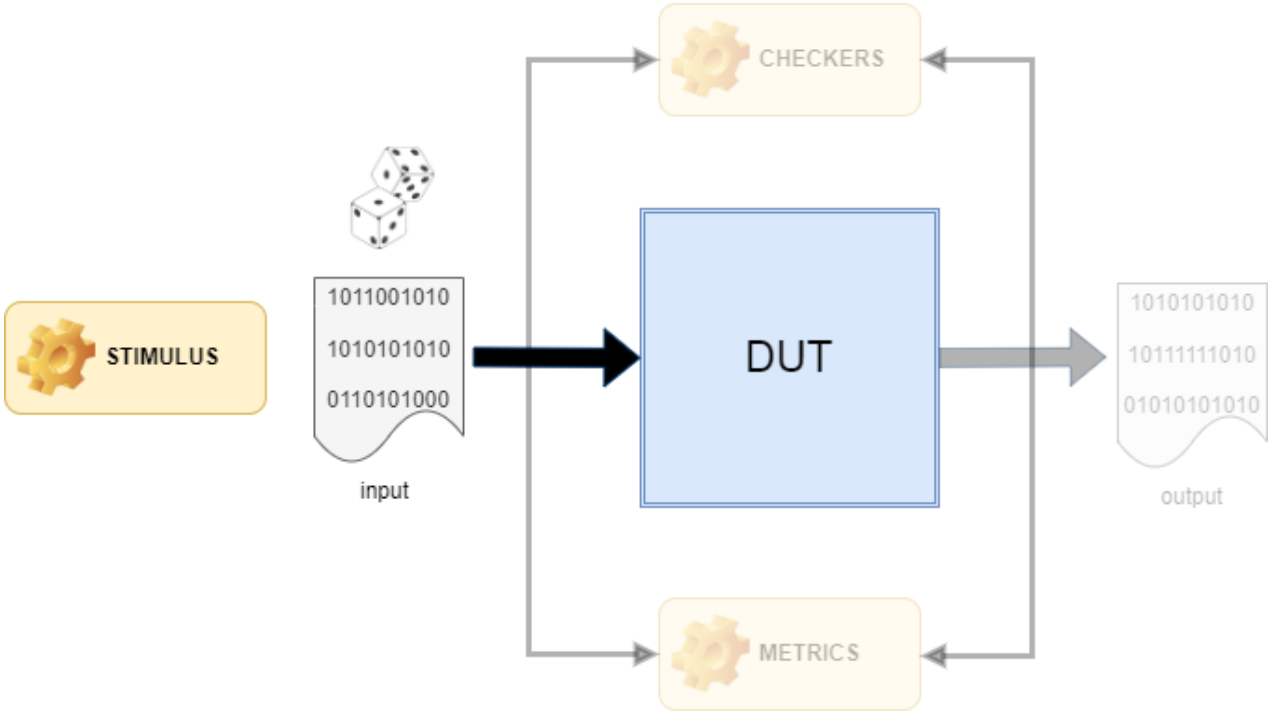
Verifying our edge detector

Edge Detector State Space

- Design: Edge detection in image pipeline
- Design state space:
  - Flops: 685
- Stimulus state space
  - 34-bits of configuration
  - 24-bit RGB data.
- Total: 2**685 x 2**58 = 2**743



Stimuli


CHECKERS
STIMULUS
1011001010
1010101010
0110101000
input
DUT
1010101010
10111111010
01010101010
output
METRICS

Edge Detector – Image size

- Support different pixel resolutions
  - Large / Small
  - Wide / Tall
- Configuration
  - X Size
  - Y Size
  - 16x16 -> 4096x4096









Edge Detector – Cropping

> Possible to crop an area of the image

> Configuration
  - Two points to represent the cropping.
    - Top left (x0, y0)
    - Bottom right (x1, y1)
    - Within the image

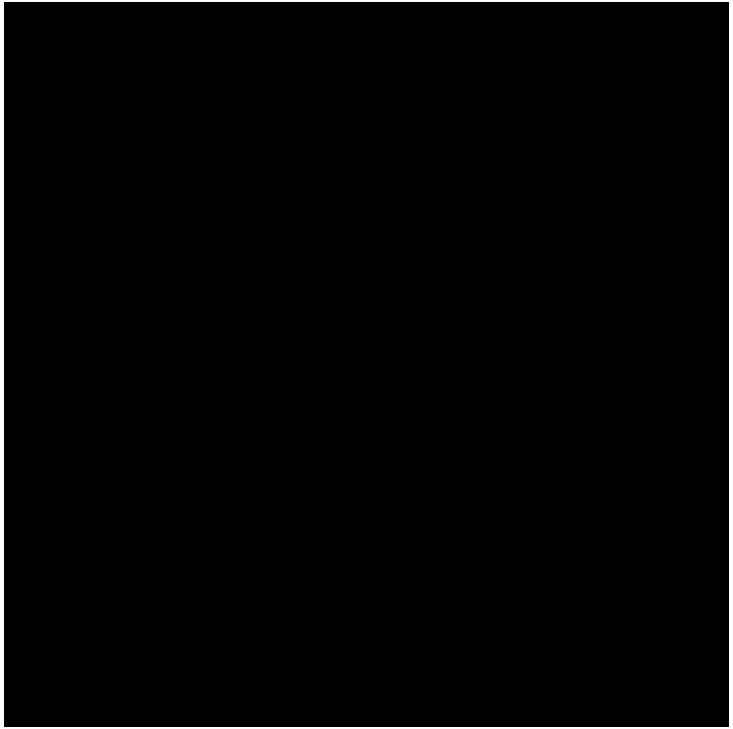


Edge Detector – Defining the stimuli space

```
1  rand int unsigned x_size, y_size;
2                    x0,     y0,
3                    x1,     y1;
4
5  constraint c_sizes {
6    x_size inside {[16:4096]};
7    y_size inside {[16:4096]};
8  }
9
```

```
10 constraint c_cropping {
11   x0 < x_size;
12   x1 < x_size;
13   x0 <  x1;
14
15   y0 < y_size;
16   y1 < y_size;
17   y0 <  y1;
18 }
```

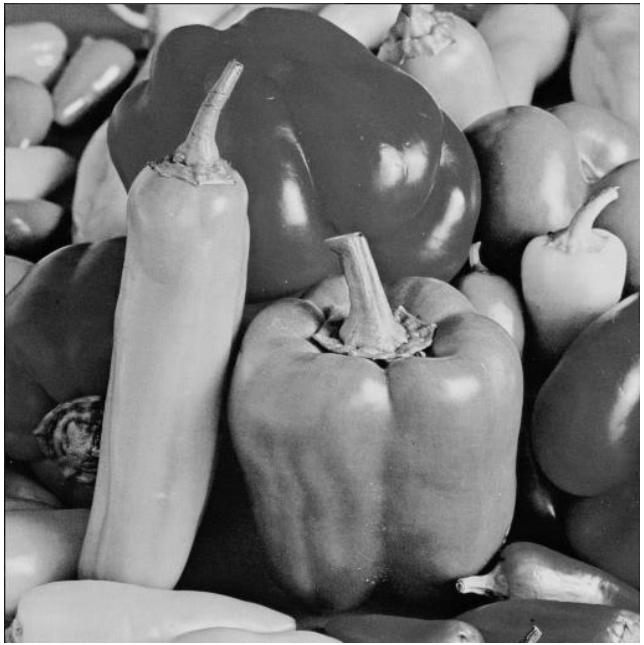
Edge Detector – Cropping Visualization



Edge Detector – Grayscale

- RGB to Grayscale conversion
  - Color to Luminance
- Y = 0.2126R + 0.7152G + 0.0722B

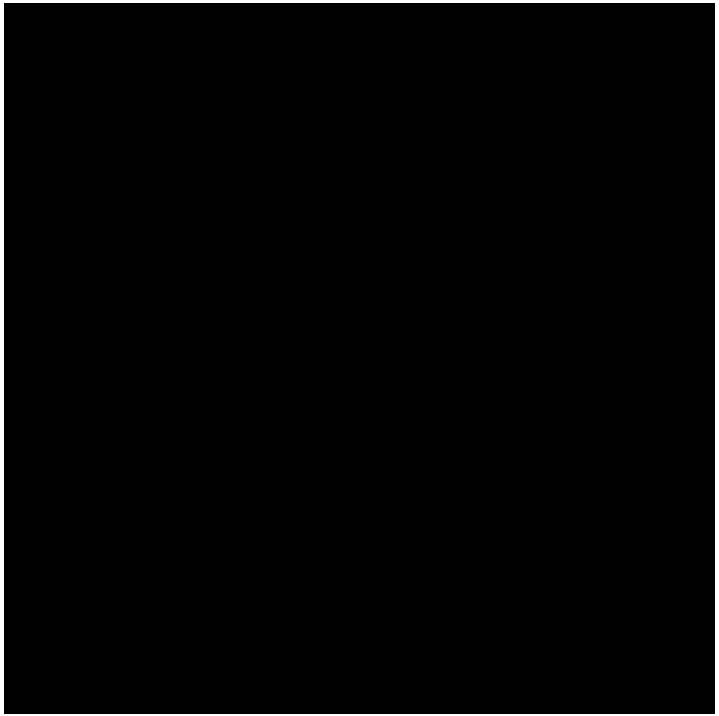
- Configuration
  - Coefficients
    - C_r | C_g | C_b
  - The sum of the coefficients should be 1



Edge Detector – Defining the stimuli space

```
1  rand int unsigned r_c, g_c, b_c;
2
3  constraint c_coefficients {
4    r_c inside {[0:10000]};
5    g_c inside {[0:10000]};
6    b_c inside {[0:10000]};
7
8    (r_c + g_c + b_c) == 10000;
9  }
```

Edge Detector – Grayscale Visualization



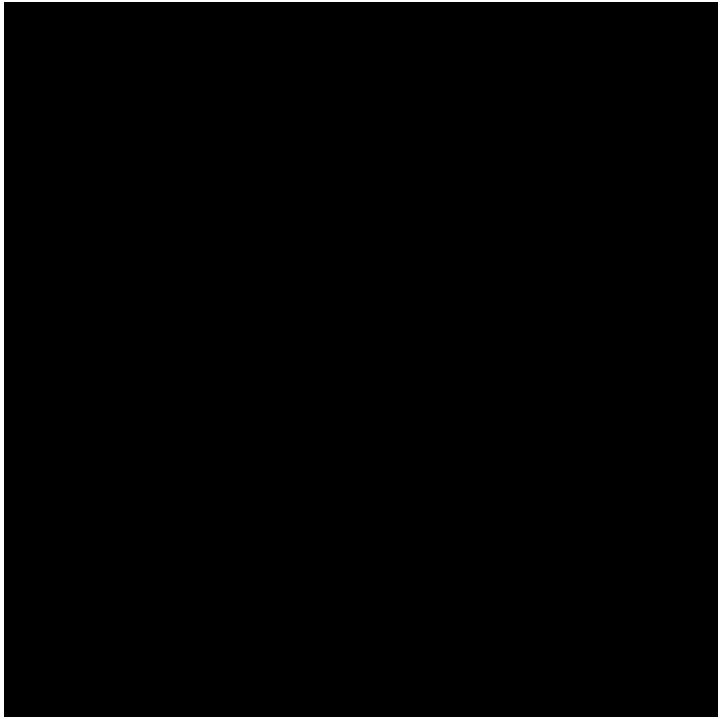
Edge Detector – Defining the stimuli space

```
1  class Cfg;
2
3    rand int unsigned x_size, y_size;
4    rand int unsigned x0, y0, x1, y1;
5    rand int unsigned r_c, g_c, b_c;
6
7    //...
8    //constraints
9    //...
10
11 endclass : Cfg
```

Edge Detector – Defining the stimuli space

```
1  class Image;
2    // Three color components
3    enum {RED, GREEN, BLUE} colors_e;
4    // Unknown number of pixels
5    rand int unsigned pixels[][colors_e];
6
7    constraint c_pixels {
8      foreach(pixels[i]) {
9        foreach(pixels[i][color]) {
10         pixels[i][color] inside {[0:255]};
11       }
12     }
13   }
14
15 endclass : Image
```

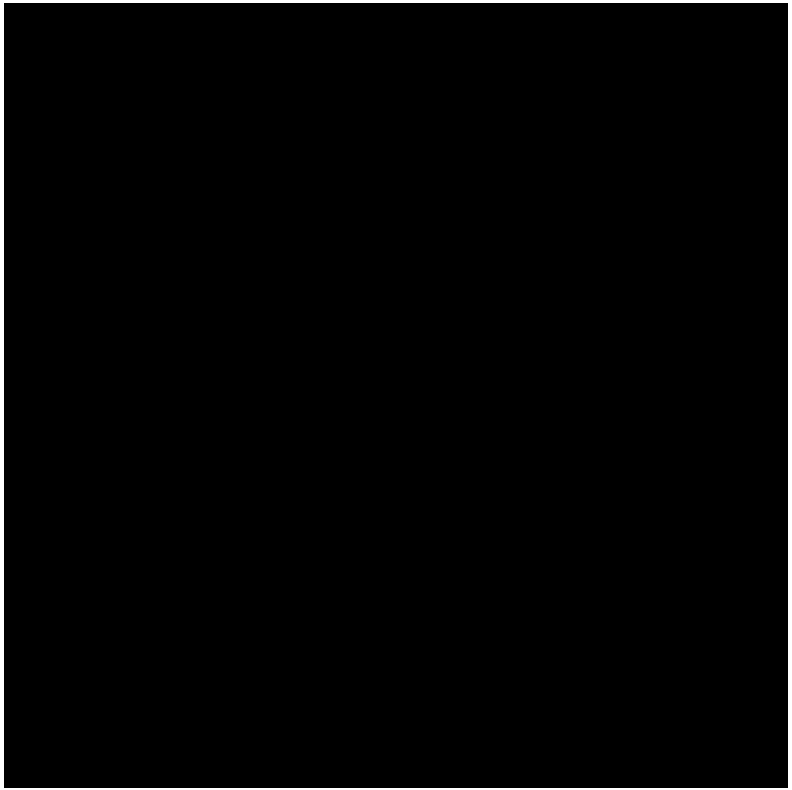
Edge Detector – Pixels Visualization



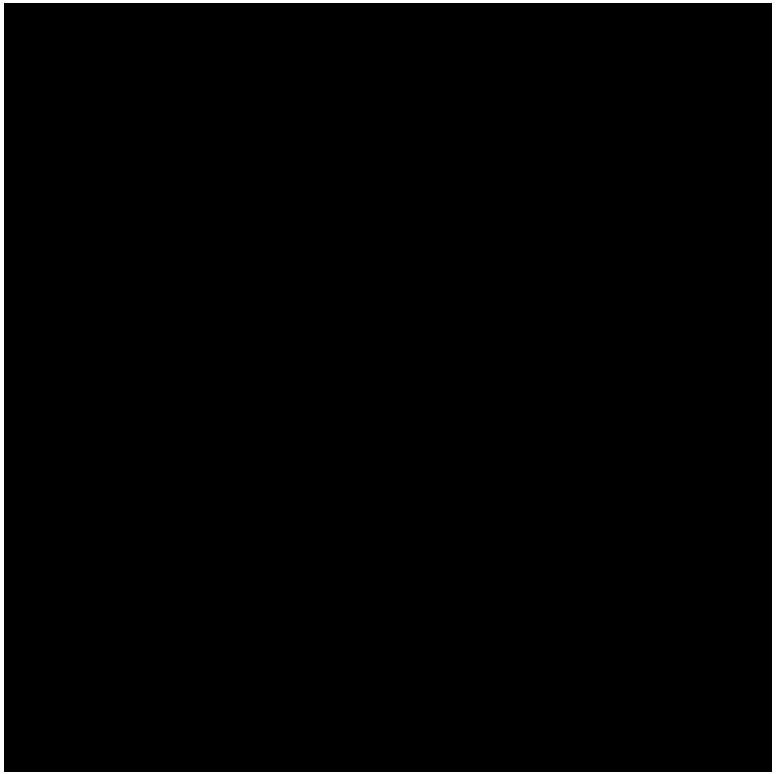
Edge Detector – Defining the legal space

```
1  class Stimuli;
2
3    rand Image img;
4    rand Cfg cfg;
5
6    function new();
7      img = new();
8      cfg = new();
9    endfunction : new
10
11   constraint c_link_size {
12     img.pixels.size == (cfg.x_size *
13                         cfg.y_size);
14   }
15
16 endclass : Stimuli
```

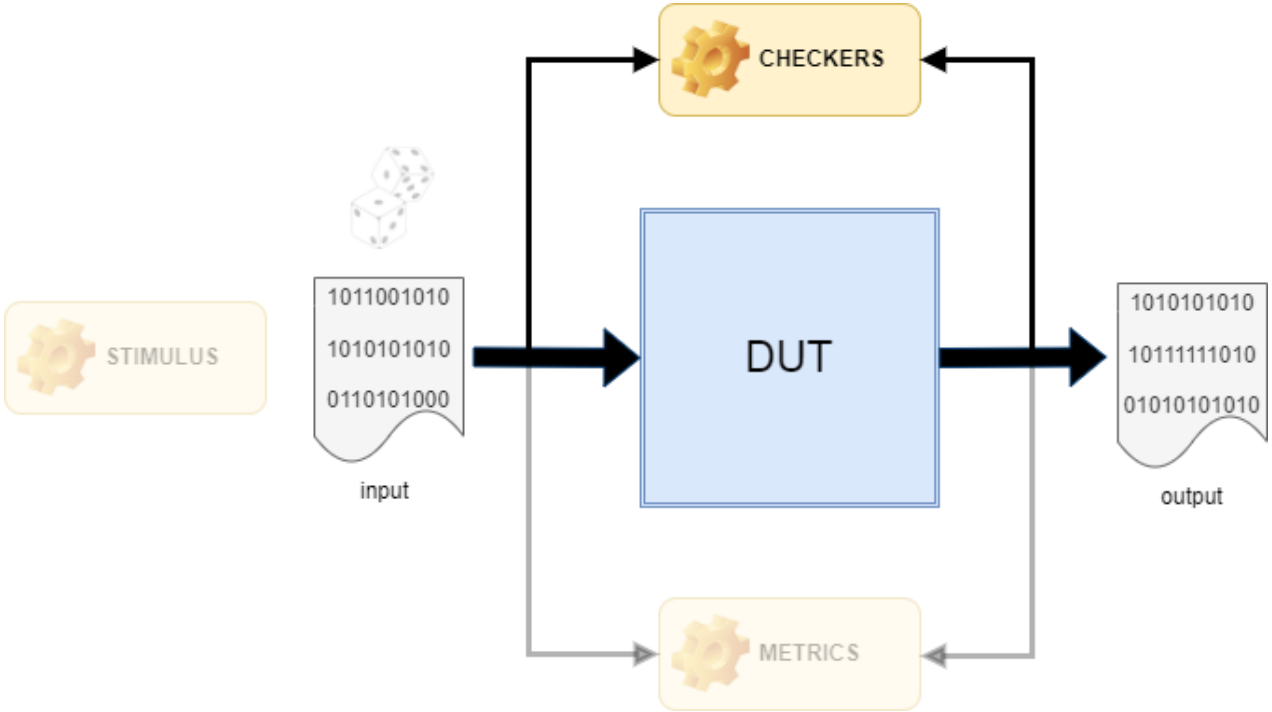
Edge Detector – Visualization



Edge Detector – Visualization



Checkers


CHECKERS
STIMULUS
1011001010
1010101010
0110101000
input
DUT
1010101010
10111111010
01010101010
output
METRICS

Checking for correctness – Reference model

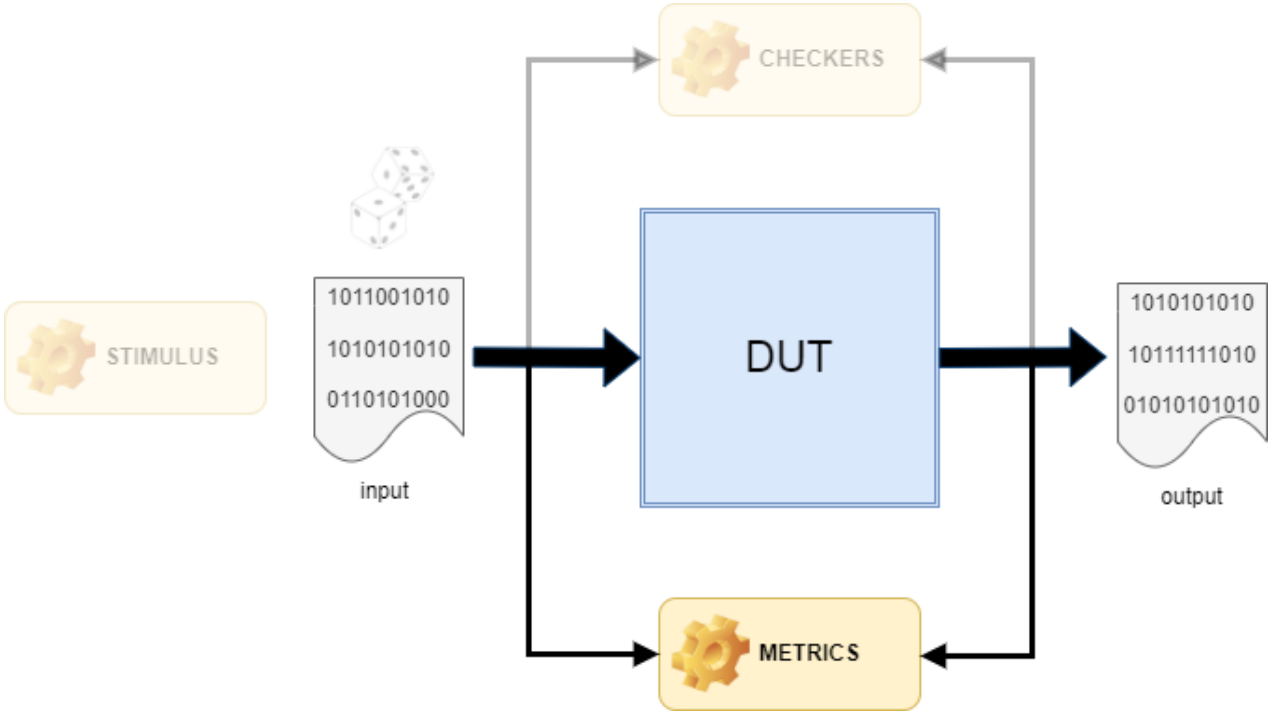
- Compare the result with a golden model
  - A reference model of the design
  - Higher-level abstraction
    - C/C++
    - Python
- This golden model is simpler
  - No need to model
    - Time dependency
    - Transactions
  - Data in -> Data out

Checking for correctness – Reference model

```
28  def crop(self, pxls):
29      pxls = pxls[self.cfg['y0']:self.cfg['y1']+1]
30      pxls = pxls[:, self.cfg['x0']:self.cfg['x1']+1]
31      return pxls

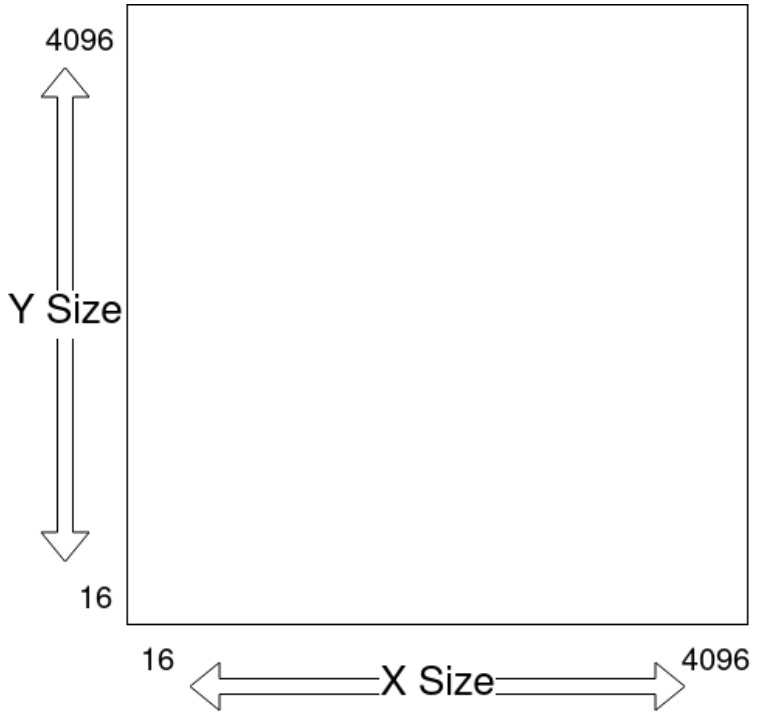
33  def grayscale(self, pxls):
34      rc = self.cfg['C_r']/10000
35      gc = self.cfg['C_g']/10000
36      bc = self.cfg['C_b']/10000
37      c = np.array([rc,gc,bc])
38      pxls = np.matmul(pxls, c)
39      pxls = pxls.round().astype(np.uint8)
40      return pxls
```

Metrics


CHECKERS
STIMULUS
1011001010
1010101010
0110101000
input
DUT
1010101010
10111111010
01010101010
output
METRICS

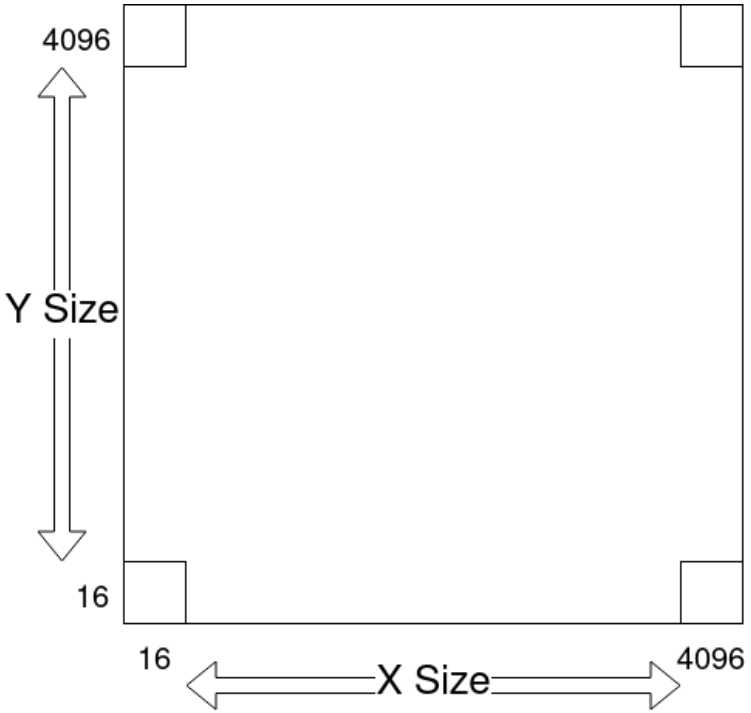
Edge Detector – Coverage

> Identify interesting scenarios



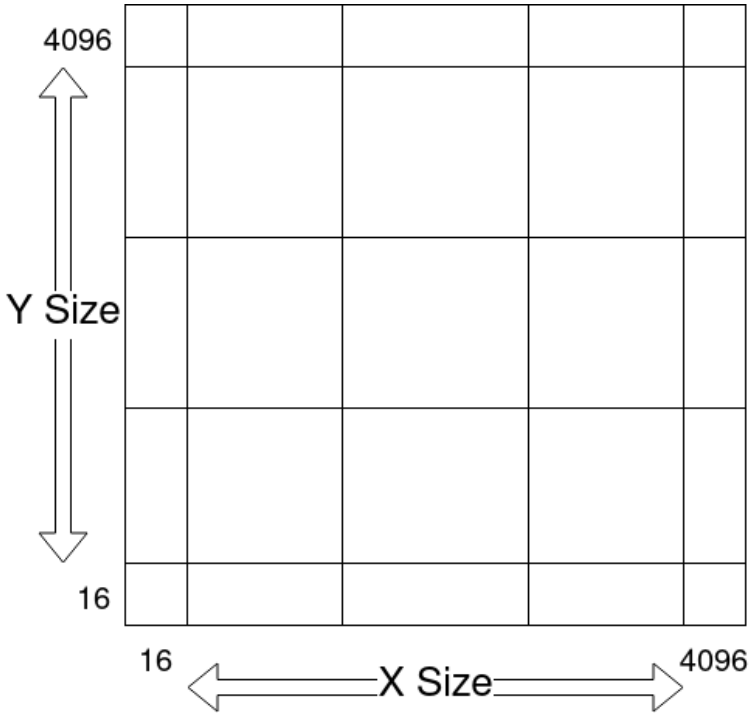
Edge Detector – Coverage

- Identify interesting scenarios
  - Smallest Image 16 x 16
  - Largest Image 4096 x 4096
  - Widest Image 4096 x 16
  - Tallest Image 16 x 4096



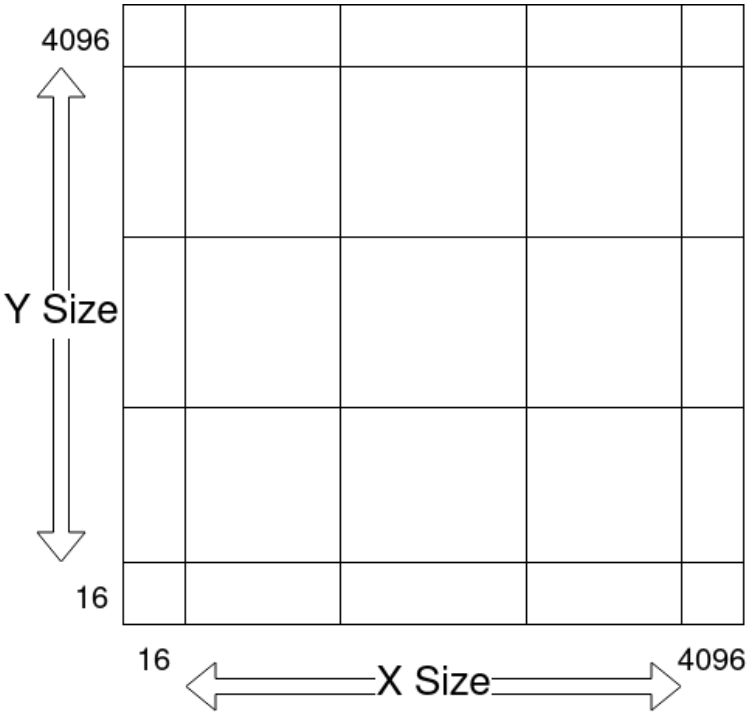
Edge Detector – Coverage

- Identify interesting scenarios
  - Smallest Image 16 x 16
  - Largest Image 4096 x 4096
  - Widest Image 4096 x 16
  - Tallest Image 16 x 4096
- Also specify some other scenarios
  - Sizes between 16-4096
  - Multiple partitions
- Coverpoints



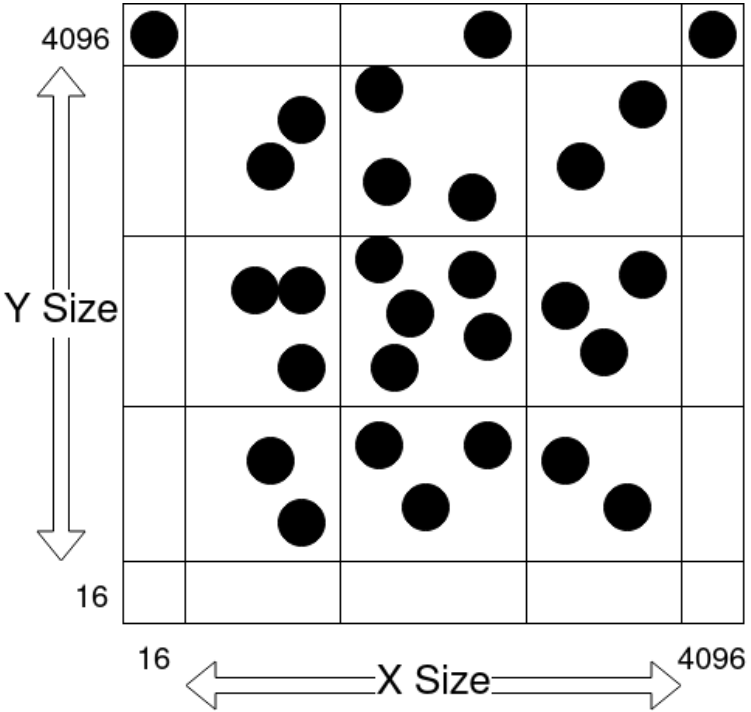
Edge Detector – Coverage

- Identify interesting scenarios
  - Smallest Image 16 x 16
  - Largest Image 4096 x 4096
  - Widest Image 4096 x 16
  - Tallest Image 16 x 4096
- Also specify some other scenarios
  - Sizes between 16-4096
  - Multiple partitions
- Coverpoints



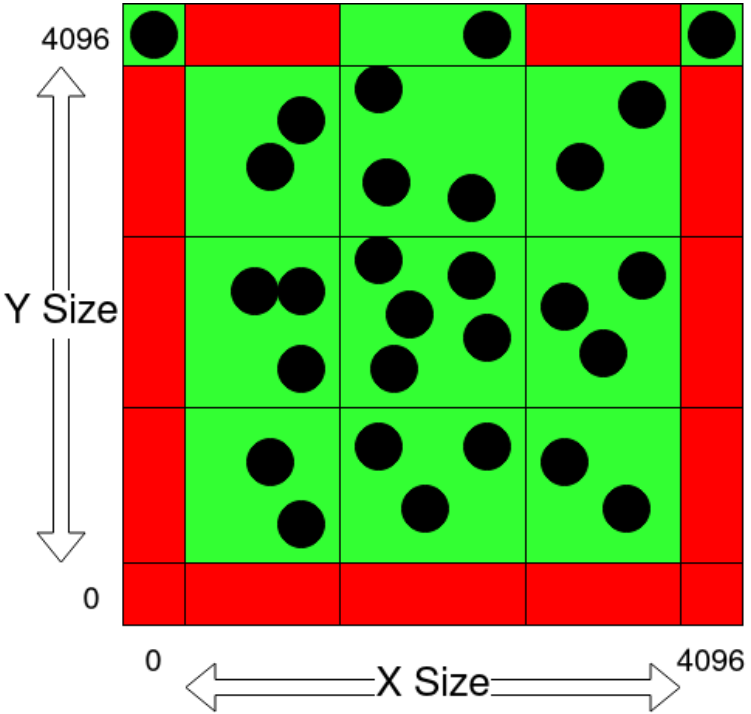
Edge Detector – Coverage

- Identify interesting scenarios
  - Smallest Image 16 x 16
  - Largest Image 4096 x 4096
  - Widest Image 4096 x 16
  - Tallest Image 16 x 4096
- Also specify some other scenarios
  - Sizes between 16-4096
  - Multiple partitions
- Coverpoints



Edge Detector – Coverage

- Identify interesting scenarios
  - Smallest Image 16 x 16
  - Largest Image 4096 x 4096
  - Widest Image 4096 x 16
  - Tallest Image 16 x 4096
- Also specify some other scenarios
  - Sizes between 16-4096
  - Multiple partitions
- Coverpoints

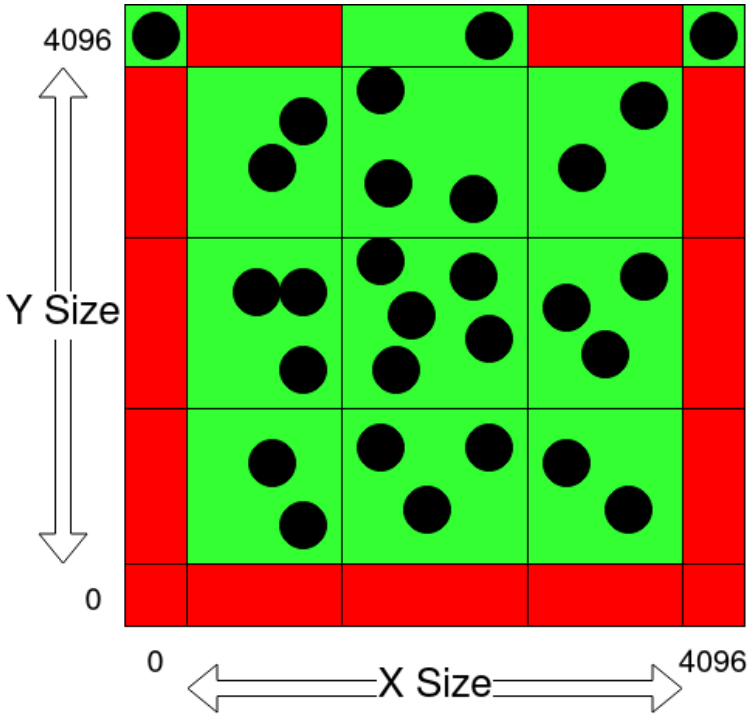


Edge Detector – Coverage

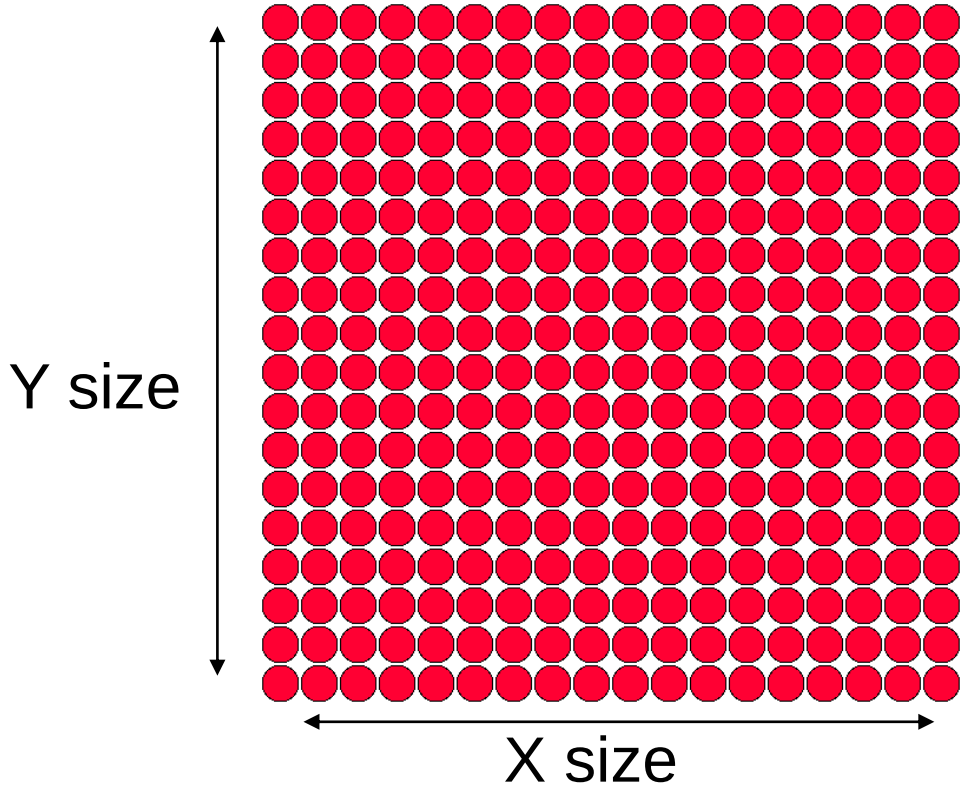
```
covergroup cg_rw_cfg;

  cp_xsize: coverpoint xsize {
    bins min[]           = {16};
    bins med[4]          = {[17:4095]};
    bins max[]           = {4096};
    illegal_bins il      = default;
  }
  cp_ysize: coverpoint ysize {
    bins min[]           = {16};
    bins med[4]          = {[17:4095]};
    bins max[]           = {4096};
    illegal_bins il      = default;
  }

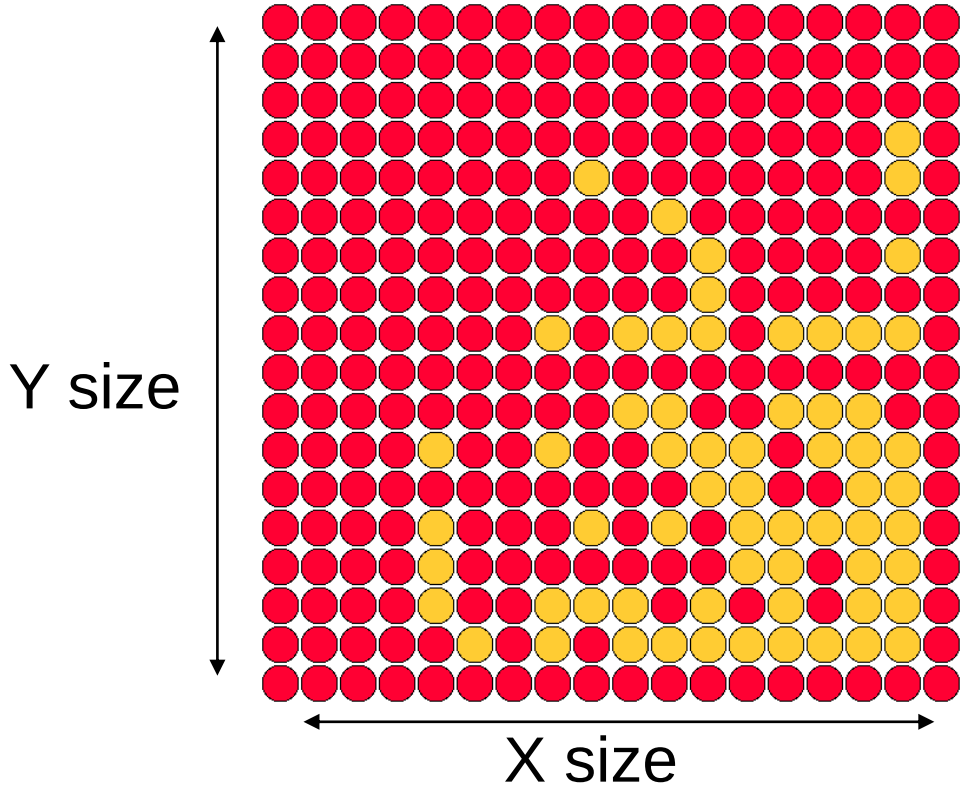
  cc_size: cross cp_xsize, cp_ysize;
```



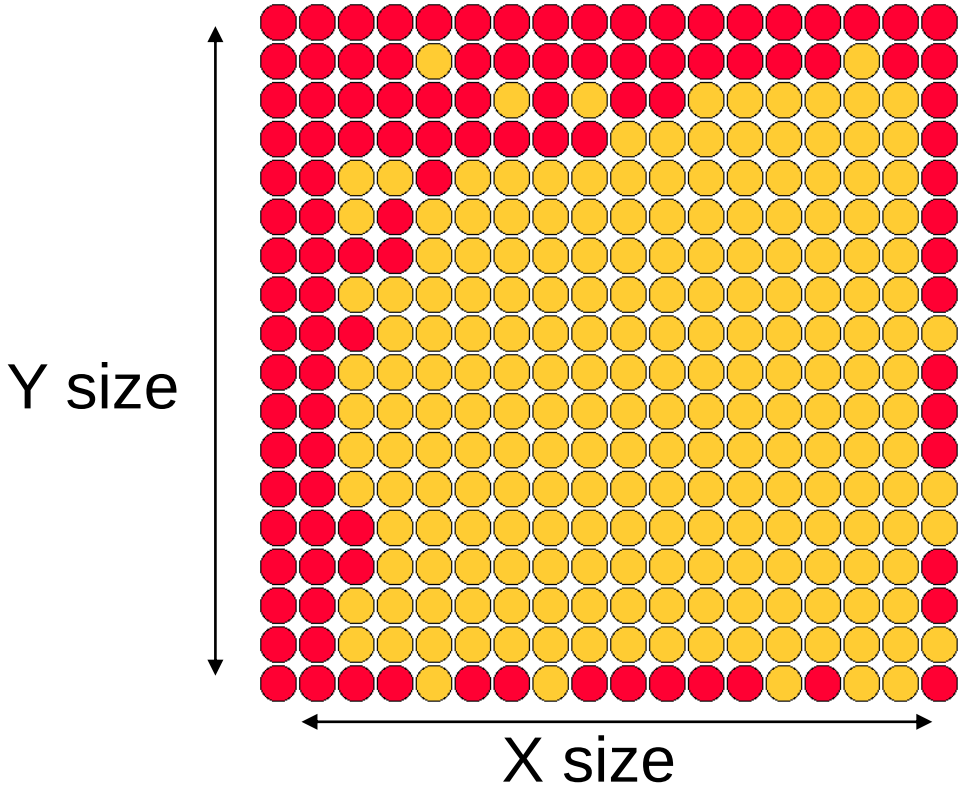
Closing coverage – Size – Initial Coverage



Closing coverage – Size – First Iteration – 100 runs

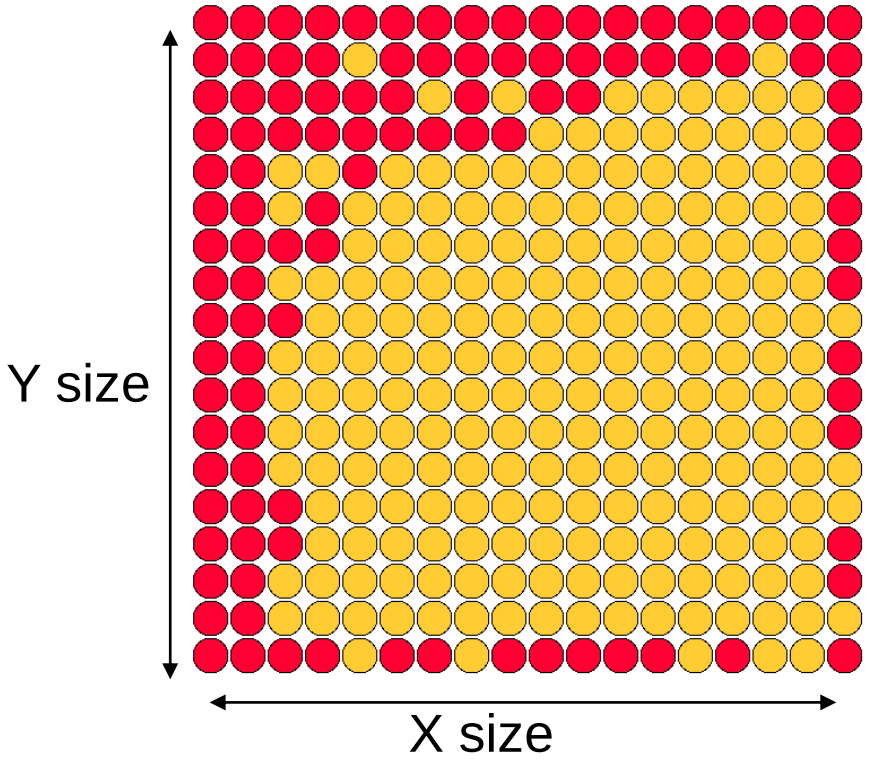


Closing coverage – Size – Second iteration – 10000 runs



Closing coverage – Distributions

> Probability of hitting corner values
  - X size
    - 16 : 1/4096
    - 4096 : 1/4096
  - Y size
    - 16 : 1/4096
    - 4096 : 1/4096

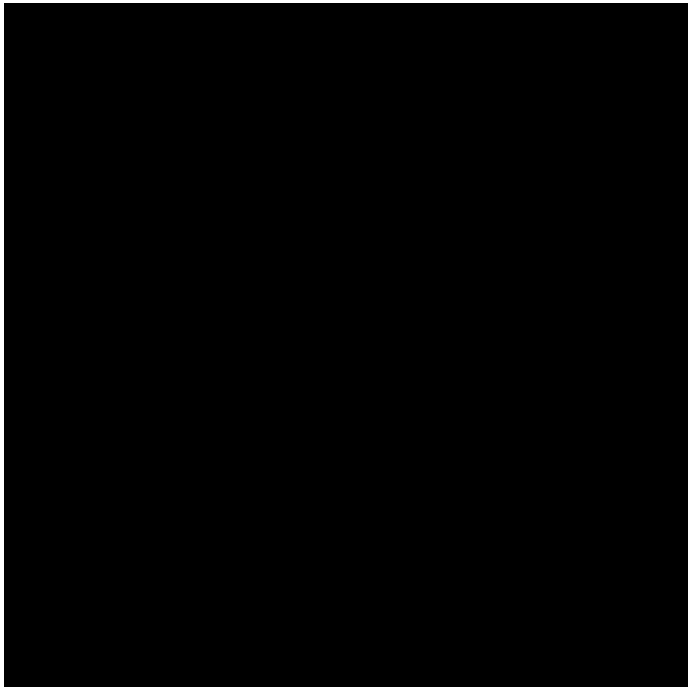


Closing coverage – Distributions

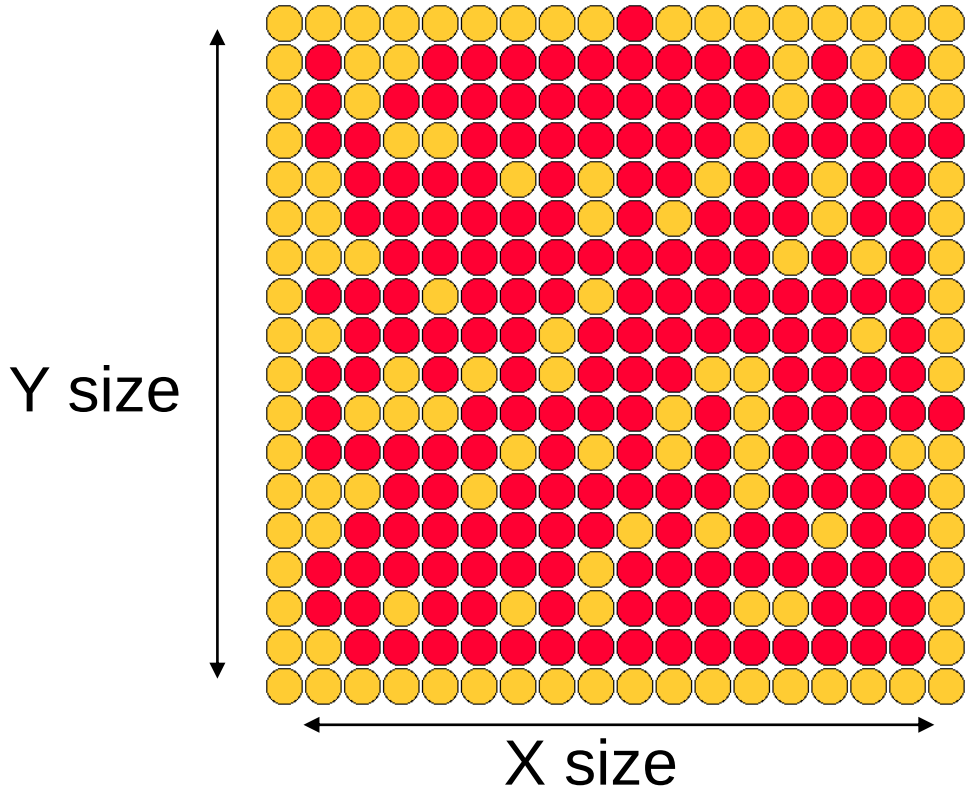
- Probability of hitting corner values
  - X size
    - 16 : 1/4096
    - 4096 : 1/4096
  - Y size
    - 16 : 1/4096
    - 4096 : 1/4096
- Dist operator puts weights on values
  - Tilts the randomization in our favor to increase coverage

```
1  class Cfg;
2
3    rand int unsigned x_size, y_size;
4    rand int unsigned x0, y0, x1, y1;
5    rand int unsigned r_c, g_c, b_c;
6
7    //...
8    //constraints
9    //...
10
11 endclass : Cfg
12
13 class Disted_cfg extends Cfg;
14
15   constraint c_size_dist {
16
17     x_size dist {16       :=33,
18                  [17:4095] :=33,
19                   4096     :=33};
20
21     y_size dist {16       :=33,
22                  [17:4095] :=33,
23                   4096     :=33};
24   }
25
26 endclass : Disted_cfg
```

Closing coverage – Distributions – 1000 runs



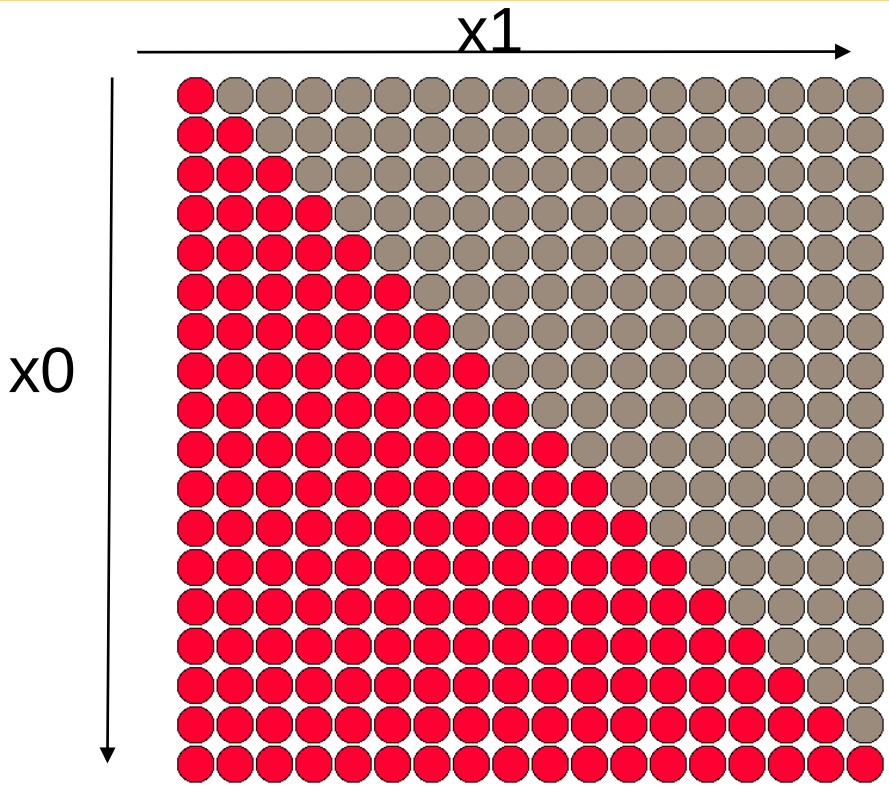
Closing coverage – Distributions – 1000 runs



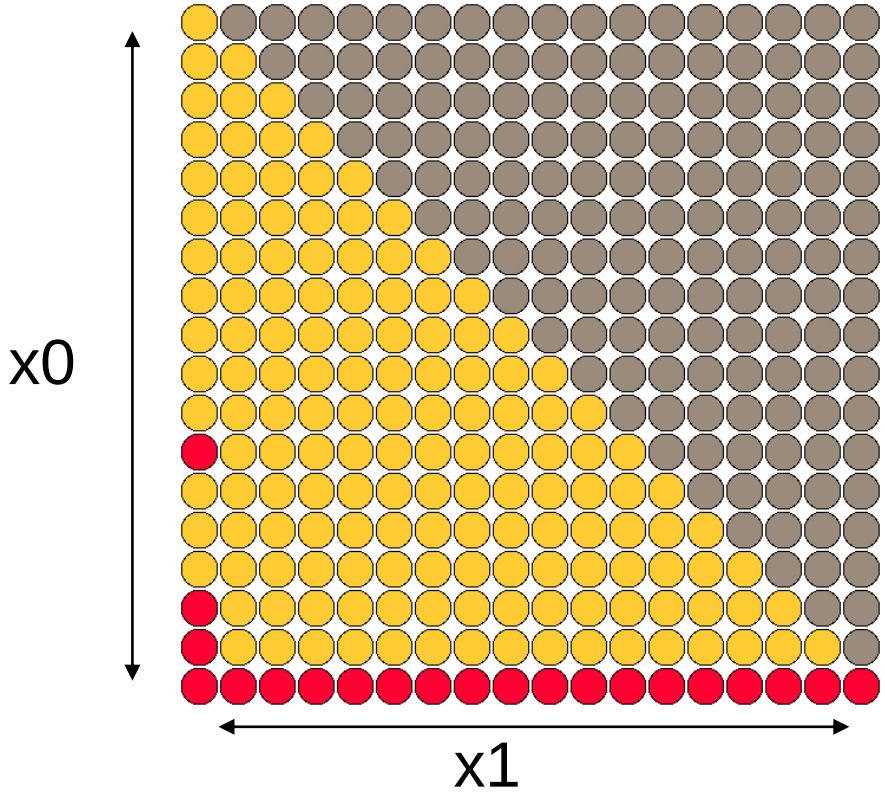
Edge Detector – Coverage

```
cc_x_crop: cross cp_crop_left, cp_crop_right {
  bins ok[] = cc_x_crop with (cp_crop_left < cp_crop_right);
}
```

Closing coverage – Cropping – Initial coverage



Closing coverage – Cropping – 10000 runs



Edge Detector – Defining the stimuli space

- Image size in x and y direction
  - Minimum 16x16
  - Maximum 4096x4096
- Cropping must be within our image
  - x0 <= x1
  - y0 <= y1

```
1  rand int unsigned x_size, y_size;
2                    x0,     y0,
3                    x1,     y1;
4
5  constraint c_sizes {
6    x_size inside {[16:4096]};
7    y_size inside {[16:4096]};
8  }
9
10 constraint c_cropping {
11   x0 < x_size;
12   x1 < x_size;
13   x0 <= x1;
14
15   y0 < y_size;
16   y1 < y_size;
17   y0 <= y1;
18 }
```

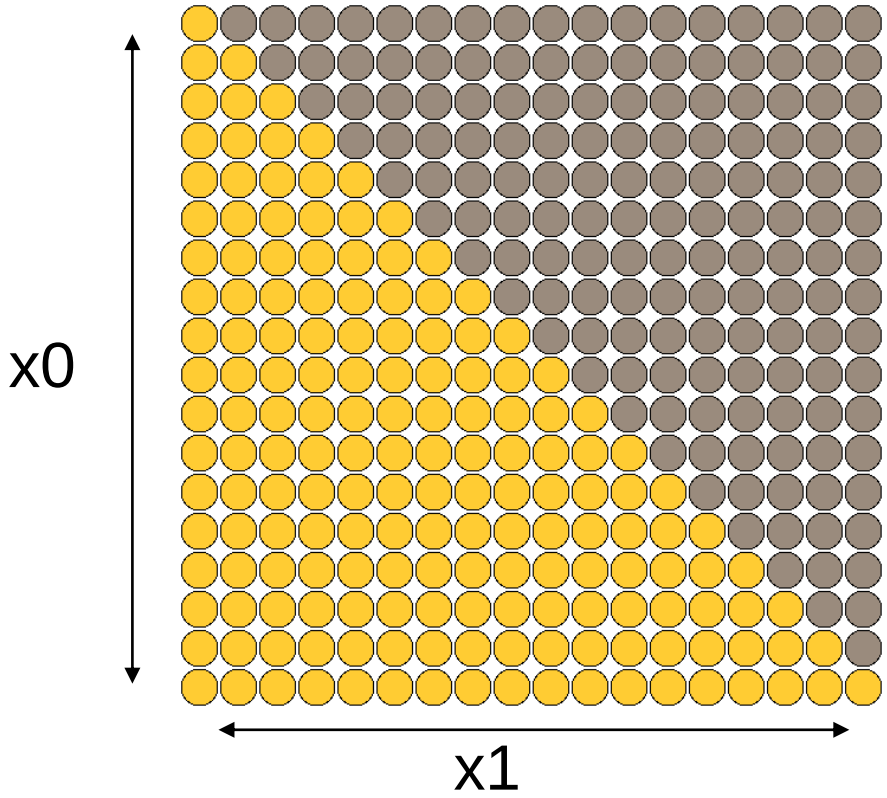
Closing coverage – Solve before

> “Solve before”
  - Variable ordering operator

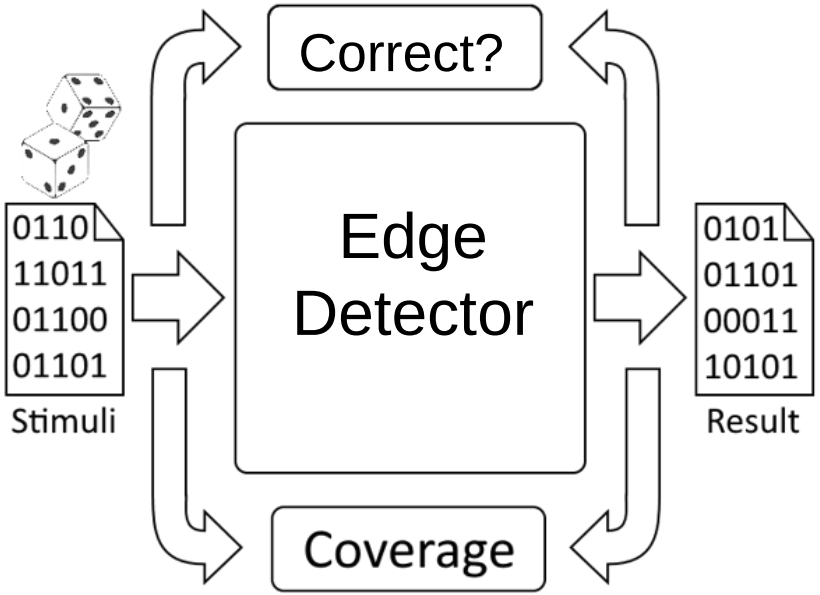
> Solve x1 before x_size
  - x1 will have an even distribution

```
1  class Cfg;
2
3    rand int unsigned x_size, y_size;
4    rand int unsigned x0, y0, x1, y1;
5    rand int unsigned r_c, g_c, b_c;
6
7    //...
8    //constraints
9    //...
10
11 endclass : Cfg
12
13 class Ordered_cfg extends Cfg;
14
15   constraint c_crop_order {
16     solve x1 before x_size;
17   }
18
19 endclass : Ordered_cfg
```

Closing coverage – Solve before – 10000 runs



Coverage-driven Constrained Random Verification



CRV trends and challenges

CRV

- Consider …
  - Cost vs. risk
- Problems ...
  - Larger and more complex
    - Checkers
    - Coverage models
    - Regressions
    - …
  - **Scalability**



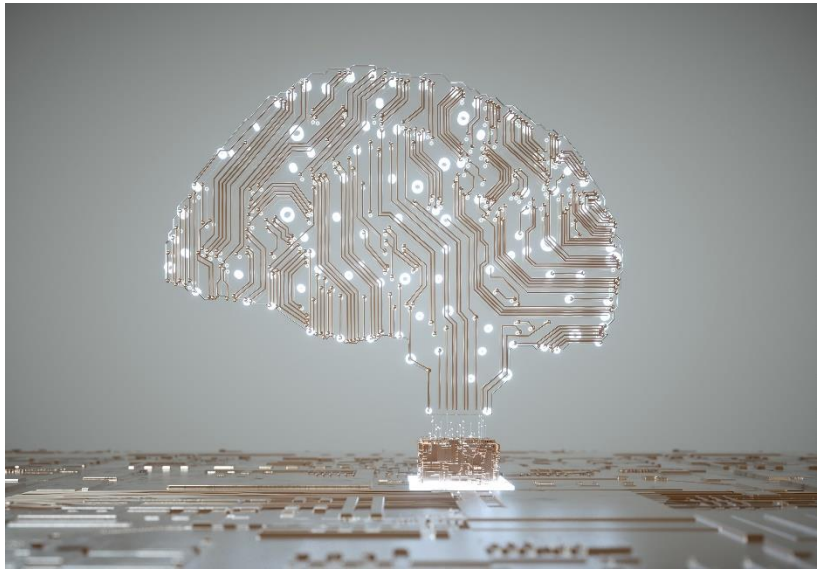
Performance

- More of the same, just faster
- Key metric in EDA tools
  - Speed
  - Parallelism
- Industry **focus** until recently



AI

- Code generation
- Bug detection and debug
- Coverage closure
  - Optimized stimuli generation
  - Regression optimization
- …



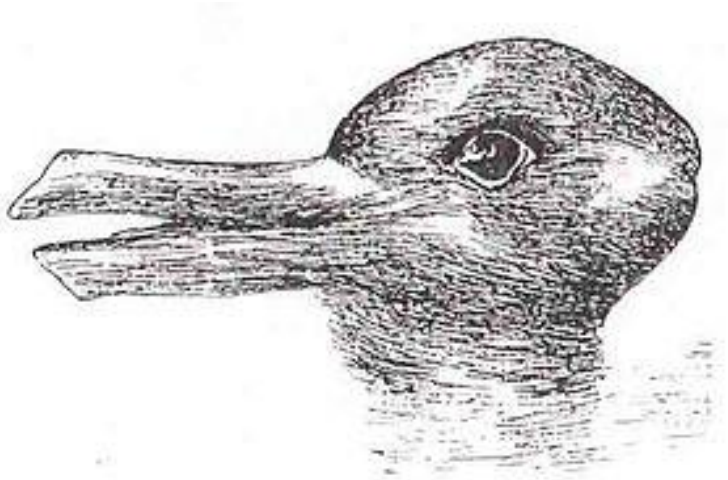
Open-source verification

- Can mean different things
  - Verification IP
  - Verification Infrastructure
  - Verification tools
    - Simulation engines
    - Formal engines
- Uptake has been slow
  - Cost vs. risk
    - Trust
    - Portability
    - Reliable support
    - Complete consistent flow
    - Ramp-up
    - Migration efforts
    - …



Paradigm shift

- Formal Methods
  - Used as early as late 1960s in software engineering.
  - Mature and mainstream technique in ASIC verification.
    - Susceptible to state space explosion.
    - Suitable for some problems
  - Active area of development
- Something else?



THANK YOU

We have more than 50 thesis proposals within:

AI	Software
Machine Learning	Algorithms
Image Quality	Radar
Audio Quality	Lidar
Supply Chain Management	Audio
UX Design	Firmware
Electronics	Deep Learning
ASIC	Image Processing
Testing	Analytics
Mechanics	Computer Vision